

Multi-Agent Decision Support for Transport Assignment

Christian van Ommeren

Kellogg College
University of Oxford



A dissertation submitted for the
MSc in Software Engineering

Abstract

This dissertation presents the design, implementation, and evaluation of a software system to enhance the capabilities of transport planners in containerised road transportation. The work addresses a critical challenge in logistics: as global trade volumes increase and supply chains become more complex, traditional manual planning approaches are becoming unsustainable, yet full automation remains impractical due to the dynamic nature of transport operations.

We develop a human-centric software solution and two distinct algorithmic approaches: a fast greedy search algorithm and a more sophisticated multi-agent system. The software solution was successfully implemented at Van Berkel Logistics, where it provides insight and decision support on a daily basis. The algorithms developed show the potential to reduce the planning time from more than 5 hours to less than 2 minutes while improving the solution quality scores from 57% to 89%.

Key contributions include: (1) a comprehensive data model that captures the complexities of real-world transport planning, (2) a scalable architecture that integrates with existing enterprise systems, and (3) a comparative analysis of centralised versus distributed algorithmic approaches. The work demonstrates that while complete automation may not be desirable, algorithms can significantly enhance planner capabilities by handling routine optimisation while preserving human oversight for data management and strategic decisions.

The research has direct practical impact, leading to the creation of Squid, a commercial software company now serving multiple logistics providers. The findings suggest that the future of transport planning lies not in replacing human expertise but in enhancing it with tools that reduce cognitive load and improve the quality of decision making.

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me throughout the course of my dissertation.

First and foremost, I am deeply grateful to my supervisor, Peter Bloodsworth, for his invaluable input and thoughtful feedback throughout this process. His guidance has been instrumental in shaping both the direction and quality of this work.

I would also like to thank my former employer, TNO, for sponsoring part of my degree. Working at TNO was not only a formative professional experience but also the place where the foundations for this research were laid.

Special thanks go to my former colleagues Jaco, Ruben, Elisah, Geert, Coen, and Yorick, the team behind Talking Trucks. Your ideas, energy, and collaboration made this journey both productive and enjoyable.

I am incredibly grateful to Michel van Dijk and Patrick Dammers of Van Berkel Logistics for their early support when I launched Squid and for providing essential data that made this research possible. I also extend my thanks to the planners of Van Berkel Logistics: Anke, Jasper, Niels, Lot, Peter, Theo, Justin, Getwin, and their supervisor Marco, for their willingness to share their expertise and their contribution to the project.

Finally, my deepest gratitude goes to my family, To Vera, for her endless enthusiasm about swimming Squids. To Marise, whose first months have coincided with the completion of this work. And above all to Francesca, whose support, patience, and encouragement made this journey possible.

Contents

1	Introduction	1
1.1	Motivation	1
1.1.1	Current challenges in Planning Container Road Transportation	1
1.1.2	Future Challenges for the Sector	1
1.1.3	Transport Planners and Their Challenges	2
1.1.4	The Need for Automation	3
1.2	Research Contribution	3
1.3	Personal Motivation	3
2	Background	5
2.1	Inland Container Transportation	5
2.2	Optimisation problem	7
2.2.1	Network Optimisation	8
2.2.2	Centralised, Decentralised, and Distributed Approaches	9
2.2.3	Greedy Search versus Multi-Agent	12
2.2.4	Large Language Models	13
2.3	Conclusion	13
3	Project	14
3.1	Scope	14
3.2	Goals	15
3.3	Requirements	15
3.3.1	Functional Requirements	15
3.3.2	Non-Functional Requirements	16
3.4	Ethical Considerations	16
3.4.1	Ethical Approval Process	17
3.4.2	Remaining concerns	17
3.5	Project Management and Planning	18
4	Design	20
4.1	Constraints	20
4.1.1	Capacity Constraints	20
4.1.2	Demand Constraints	21
4.2	Data Model	22
4.3	Architecture	24
4.4	Integration	24
4.5	Software Components	25

5	Implementation	30
5.1	Graphical User Interface	30
5.2	Postgres Database	32
5.3	Synchronisation Service	32
5.4	OSRM Routing Service	33
5.5	Geocoder Service	34
5.6	Algorithm Development	34
5.6.1	Data Transformation	35
5.6.2	Single-Agent Greedy Search	36
5.6.3	Multi-Agent System	38
6	Evaluation	42
6.1	Problem Setup and Methodology	42
6.2	Individual Driver KPIs	42
6.3	Overall Solution KPIs	43
6.4	Scoring system	43
6.5	Problem size	44
6.6	Results	44
7	Conclusion	51
7.1	Revisiting Our Goals	51
7.2	Meeting Requirements	51
7.3	Project Reflections	52
7.3.1	What Went Well	52
7.3.2	What Could Have Been Better	52
7.3.3	What I Would Do Differently	53
7.4	Key Takeaways	53
7.5	Key Takeaways	53
7.6	Future Work	54
7.7	Final Thoughts	54
A	Appendix	60
A.1	Letter of Consent	60
A.2	Decision Ethics Committee	63
A.3	Project Steps	63
A.4	Process Diagram	65
A.5	User Story	66
A.5.1	Name	66
A.5.2	Description	66
A.5.3	Actors	66
A.5.4	Preconditions	67
A.5.5	Standard Flow	67
A.5.6	Alternative Flows	67
A.5.7	Result	68
A.6	Wireframes	69
A.7	Designs	74
A.8	Application Screenshots	79

1 Introduction

1.1 Motivation

The planning of containerised road transportation is a complex and dynamic challenge. It requires coordination between a large number of variables, constraints, and real-time events. The traditional planning process is highly dependent on human expertise, which, while invaluable, makes the process slow, prone to human error, and difficult to scale as demand increases. In this project we are exploring and implementing a solution that automates parts of the planning process to increase efficiency, reduce errors, and support transport planners in their decision-making.

This is not an easy task. In fact, experienced human planners are quite capable of creating and monitoring a transport planning. They manage to keep a surprisingly large amount of information in their head while responding to continuous triggers in a time pressured work environment. Heuristics and even more complex algorithms have a hard time beating their planning solutions [33].

In this chapter, we will talk the reader through the current and future challenges of transport planners and the logistics sector in general. We will address how this research contributes to the larger body of knowledge and conclude with a personal motivation.

1.1.1 Current challenges in Planning Container Road Transportation

Planning containerised road transportation involves balancing numerous constraints, such as truck and chassis weight limits, driver rest times, and environmental regulations. This occurs in a highly dynamic environment, with unexpected factors such as road congestion, handling delays at port terminals, deviations in sea vessel arrivals, and last-minute bookings or cancellations. As the volume of goods transported increases [14], the complexity of the planning process also increases, making it harder to keep track of all the variables involved.

*“Planning is much like solving a puzzle
where the pieces keep changing, both in number and form.”*

In addition to complexity and a dynamic environment, much of critical information is not digitally available or is scattered across disparate sources such as websites, IT systems, spreadsheets, or the minds of planners, drivers, and process operators. This creates a fragmented and inefficient system that requires constant updates, manual checks, and ad hoc problem solving.

1.1.2 Future Challenges for the Sector

The transportation industry faces additional future challenges that require more efficient planning. According to the European Environment Agency [10] transport-related green house gas (GHG) emissions have increased by more than 25% since 1990, where other sectors have generally seen reductions. With mandatory CO₂ reporting on the horizon in Europe [11], the need

for efficient and sustainable planning will become more pressing. Electric trucks, while part of the solution, come with their own set of complications, particularly around charging times and high initial costs. Gaining efficiency without sophisticated algorithms will be increasingly difficult.

Recent global events have further highlighted the strategic importance of the logistics sector. During the COVID-19 pandemic, logistics was designated as a critical infrastructure sector, underscoring its essential role in maintaining societal and economic functions. Disruptions such as the Suez Canal blockage in 2021 [6] and, more recently, the effects of new US import tariffs [32], have shown how vulnerable global supply chains can be. Resilience and adaptability are equally important as efficiency.

In the aftermath of COVID-19, ports around the world experienced severe congestion, with hundreds of container ships waiting offshore for days or even weeks. These backlogs led to cascading delays, shortages of goods, and increased costs for businesses and consumers alike. According to the World Bank and IAPH [2], global container port congestion peaked in 2021, with some ports operating at more than 90% capacity for extended periods.

The consequences of such backlogs are far-reaching: manufacturers face production stoppages due to missing components, retailers struggle to keep shelves stocked, and transport companies incur significant demurrage and detention fees. For transport planners, these disruptions create a complex puzzle of rescheduling deliveries, managing detention costs, and reallocating resources. These tasks become overwhelming when handled through manual planning processes alone.

The effect of disruptions does not only signal the importance of the logistics sector for the global economy, but also underscore the need for more resilient, efficient, and automated planning systems in container logistics. Another worrying longer-term trend is the shrinking labour market, both in terms of available drivers and planners [22][39]. To stay competitive, the level of digitisation and efficiency has to improve. It is crucial to support planners in making better decisions while keeping drivers satisfied and engaged. Planning systems must evolve to handle both everyday complexity and unexpected disruptions, ideally in a way that reduces human dependency while preserving operational flexibility. The ability to quickly adapt to disruptions and optimise resource allocation is essential to maintaining the flow of goods given the dynamic nature of modern logistics.

1.1.3 Transport Planners and Their Challenges

The role of transport planners is critical to the effective operation of containerised logistics. According to the Dutch Sector Institute for Transport and Logistics (STL) [9], transport planners are responsible for developing and managing transport schedules, ensuring regulatory compliance, optimising operational efficiency, and maintaining communication with key stakeholders such as drivers, customers, and service partners.

Three key characteristics of the job, provided by STL [9], include:

1. **Role and responsibilities:** Planners coordinate driver assignments and ensure that transports comply with safety, legal, and environmental regulations. They must balance multiple objectives: on-time delivery, efficient use of transport capacity, and driver satisfaction.
2. **Working environment:** The work environment is dynamic and time-pressured. Planners are required to respond quickly to disruptions, such as delays or last-minute changes, which makes the role inherently stressful.

3. **Skills and competencies:** Effective planners need strong problem solving abilities, organisational skills, and clear communication. They must also be adept at using digital tools and handling multilingual communication.

In summary, transport planners must balance complex operational constraints, regulatory compliance, and stakeholder communication while operating in a high-pressure environment where decisions have immediate financial and operational consequences. Their role requires both tactical problem-solving skills and strategic oversight, making it an ideal candidate for algorithmic support.

1.1.4 The Need for Automation

Given the complexity and the time-pressured work environment, transport planners often make quick decisions based on experience and established processes. There is little time and headspace to consider the larger picture and make decisions towards an optimisation goal. The resulting schedules may be feasible but are not necessarily the most efficient in terms of capacity utilisation or cost-effectiveness. This is where automation comes in. By automating parts of the planning process, and particularly the initial stages of capacity planning, transport companies can reduce the cognitive load on planners, increase efficiency, and allow planners to focus on higher-level decision-making and problem solving.

1.2 Research Contribution

This research builds on previous work in the field, specifically at TNO [43][33][42][5], where research has been carried out on algorithms to support planners of containerised road transportation. This research has laid the groundwork, but did not fully address the real-world complexities of transportation planning. This dissertation aims to refine and extend that existing body of knowledge by making a full software solution that can be applied in a real-world scenario.

1.3 Personal Motivation

Having worked for five years at TNO as a research scientist in Sustainable Transport and Logistics, the author has had the opportunity to study various optimisation algorithms and their application in the transportation sector. My involvement in research on decentralised logistics and autonomous trucks through the *Talking Trucks* project deepened my interest in the future of logistics and transportation, and laid the foundation for starting my own company: Squid.

Squid aims to help the planners of containerised road transportation by:

- Digitising missing information;
- Providing insight into transport capacity and demand;
- Automating basic steps in the planning process;
- Alerting planners when problems are likely to occur.

Van Berkel Logistics is Squid's launching customer. The collaboration provided real data, human expertise, and the opportunity to develop the software presented in this dissertation. This

project describes and reflects upon the decisions made along the way and explores opportunities for further improvement.

The reason Squid is called Squid is not because having a friend with eight arms helps you to get your work done faster. The real inspiration came from biology: Squids have a partially distributed brain. While the central brain coordinates their actions, each tentacle contains its own neurons and can operate semi-autonomously [16]. Could we design a software solution that draws inspiration from this amazing biological feat?

2 Background

In this project we are specifically looking at optimising road transportation for the network of a group of inland container terminals in the Netherlands: Van Berkel Logistics.

While this is a limited scope, the challenges and solutions discussed are representative of containerised road transportation worldwide. The core issues - balancing capacity, regulatory compliance, dynamic disruptions, and the need for efficient assignment - are almost ubiquitous in the logistics sector, whether at a major European port, an American rail hub, or an Asian transshipment centre. By studying this real-world environment, we aim to develop and validate methods that are applicable in the broader domain of container logistics.

The second reason why we focus on this specific group of inland terminals is that they were the launching customer of Squid. Their continued support has allowed us to spend close to ridiculous amounts of time on researching, designing, and implementing the software components presented in this dissertation.

First, we will guide the reader through some background on inland container transportation to be able to comprehend the domain. Next, we will clarify the optimisation problem, discuss some high-level approaches to solving it, and conclude with the learnings that will lead us into the formulation of our requirements and ultimately the design and implementation of a solution.

2.1 Inland Container Transportation

A container is never an isolated move from A to B. The typical *inland* trajectory of a shipping container is visualised below in Figure 2.1:

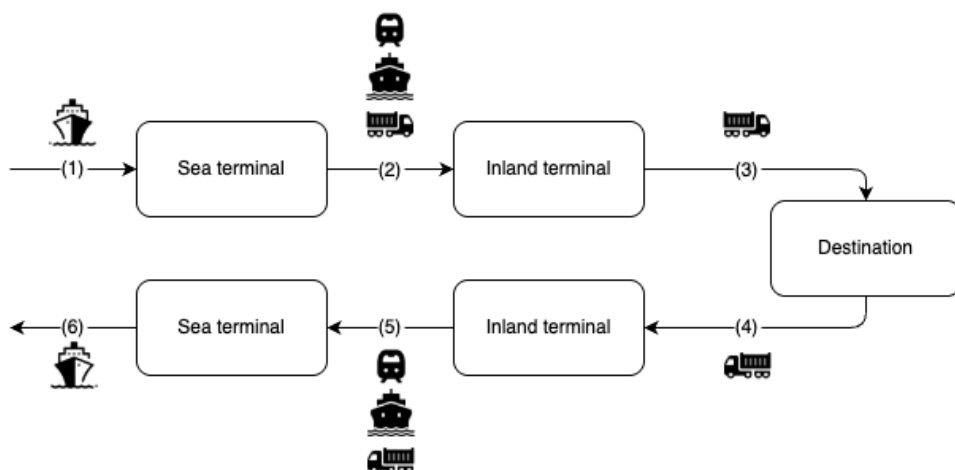


Figure 2.1: A typical container roundtrip

1. Arrives at a sea terminal by ship
2. Travels to an inland terminal by truck, train or barge (inland ship)

3. Travels to its destination by truck
4. Returns to an inland terminal by truck
5. Returns by truck, train or barge to the sea terminal
6. Leaves the sea terminal by ship

The reason for this roundtrip is that the container is usually owned by the shipping line. Each container receives a turnaround window and must be returned to the shipping line within that time frame. There are, however, multiple exceptions:

- A planner may decide to truck a container straight from the destination to the sea terminal. A good reason for this is when the container will not reach the sea terminal in time by barge to leave with an ocean-going container ship.
- A container may not return to the sea terminal at all. In this case it is kept in stock at the inland terminal. Inland terminals have agreements with some shipping lines that allow them to keep some stock at hand for later export reuse.
- The cargo may be discharged from a container at its destination, then moved to a different address for loading. This is essentially a merge of an import roundtrip with an export roundtrip.

The network of Van Berkel Logistics which we will serve as an example in this project is shown in Figure 2.2. Their service proposition to clients is to take care of the inland transportation of their containerised cargo.

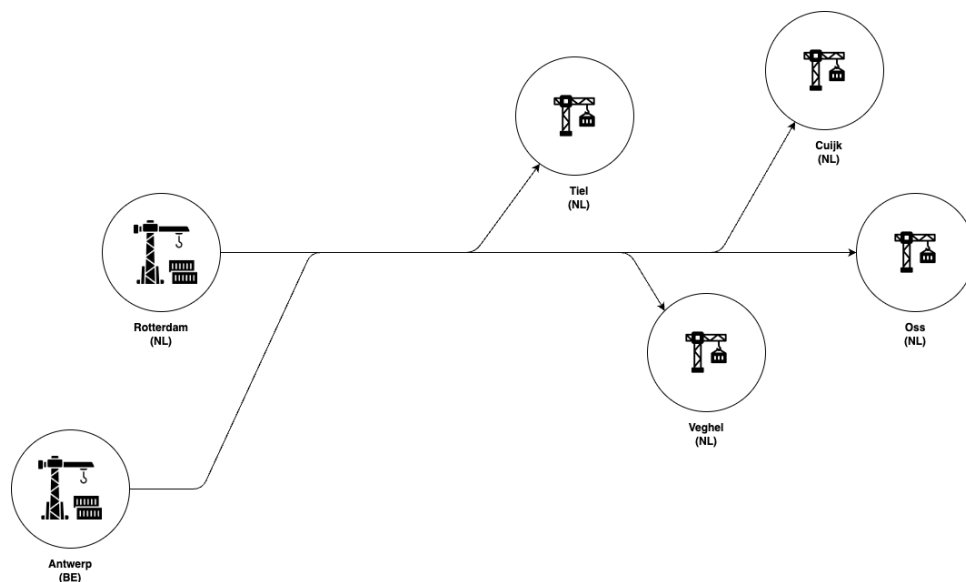


Figure 2.2: A typical container roundtrip

To move containers, Van Berkel Logistics operates a fleet of barges and trucks. Containers are transported by barge from the sea terminals in Rotterdam and Antwerp to one of their inland terminals in Cuijk, Oss, Tiel, and Veghel. Long distance trucking only occurs upon specific request from the customer or when containers are under time-pressure. The final transportation leg to the customers always happens by truck.

2.2 Optimisation problem

The full optimisation problem is a network-based challenge in which decisions must be made regarding the mode and route of container transport. This involves choosing whether to move a container by truck or barge, and whether to deliver it directly from the sea terminal to the customer or route it via an inland terminal. These choices are interdependent: for example, if the handling slot of a barge at the sea terminal is delayed, it may become necessary to shift more containers to trucking, which, in turn, may affect delivery times at customer locations. While these intermodal interactions are critical to the overall system, this dissertation focuses specifically on the road transportation component.

Currently, human transport planners are responsible for making the decisions. Creating a transport plan can be broken down into three main tasks:

1. **Organising sufficient transport capacity:** Ensuring that enough trucks and drivers are available to handle the expected workload.
2. **Assigning transports to truck/driver pairs:** Matching transports to truck and driver combinations, taking into account factors such as timing, location, driver preferences, and legal constraints.
3. **Monitoring execution and adjusting the plan when needed:** Continuously tracking transport progress and responding to disruptions or delays by changing assignments in real time.

Organising transport capacity requires having a good forecast and picking up the phone to scale with chartered drivers. The output of this first task forms the input of the second task. The work in this dissertation does provide short-term insight into transport demand and capacity, as displayed in Figure 2.3. The task of scaling capacity resides on a more tactical level as driver schedules are determined weeks in advance. A good demand forecast, however, is not available until a few days prior to execution. A longer-term forecast is desirable but out of scope for this dissertation. The number of drivers and transports is regarded as a fixed input, but in reality a planner may attempt solving an imbalance by:

1. Asking regular drivers to work overtime;
2. Asking part-time drivers to work an additional day;
3. Asking charter companies to supply additional drivers;
4. Outsourcing transports to other transport companies;
5. Shifting delivery times in consultation with the customer;
6. Informing customers that some trips cannot be executed.

The second task, assigning transports, is the core of this project. A good transport plan must balance multiple objectives:

1. **Compliance with all rules and regulations**, this includes mandatory rest times for drivers and weight restrictions on trucks and chassis.

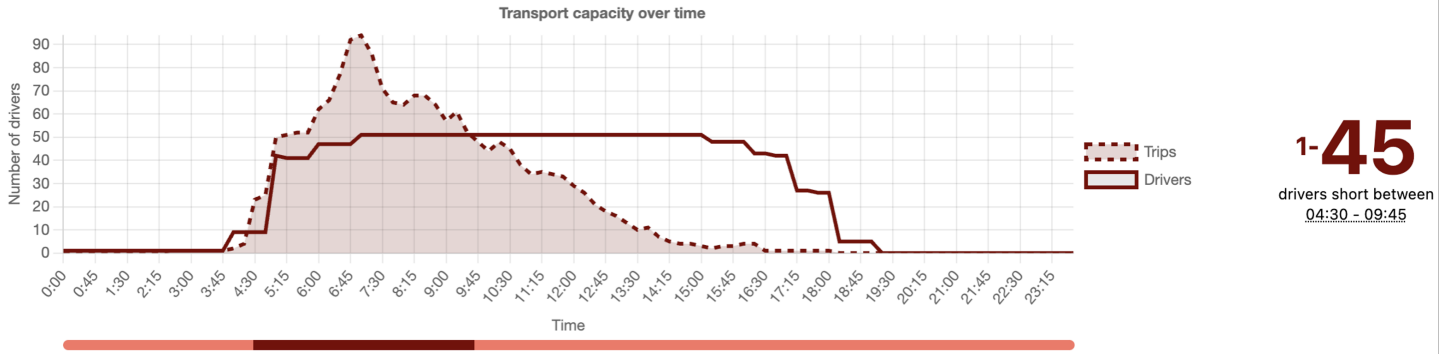


Figure 2.3: Transport capacity versus transport demand

2. **Maximise on-time performance** in order to keep customers happy and maintain a reliable flow of goods.
3. **Maximise of transport capacity utilisation** by avoiding empty driving and minimising idle time.
4. **Maximising driver satisfaction** by accounting for driver preferences and work conditions.

Compliance with legal regulations is a hard constraint. The other three are optimisation goals, we can violate on-time performance, capacity utilisation, and driver satisfaction to some extent, but doing so will incur a cost.

Finally, the third task, monitoring execution and adjusting the plan, is out of scope, but replanning can be facilitated with the algorithms developed as part of this work.

2.2.1 Network Optimisation

On a more theoretical level, we are attempting solving a network optimisation problem. Networks are often modelled as graphs, where nodes are physical objects in the network and edges describe the relationship between connected nodes. These problems are NP-complete, because there is no known algorithm that can efficiently find a solution in polynomial time [15]. This is because the number of possibilities increases exponentially with every additional node. Our average problem size consists of 53 drivers and 240 transports (see Table 6.1), resulting in such a large number of possible combinations that makes it infeasible to try all possibilities to find the optimal solution within an acceptable amount of time. The number representing the total amount of driver/transport combinations is a 418 digit number:

$$55^{240} \approx 1.38 \times 10^{417}$$

A common way to solve such problems, within acceptable time, is to apply a heuristic. Figure 2.4 displays a problem where we need to depart from A, visit all nodes, and return to A. The numbers on the edges are the minutes it takes to travel between two nodes. If we use a shortest-path heuristic, we obtain a pretty good result in less time than we would have needed to evaluate all possibilities.

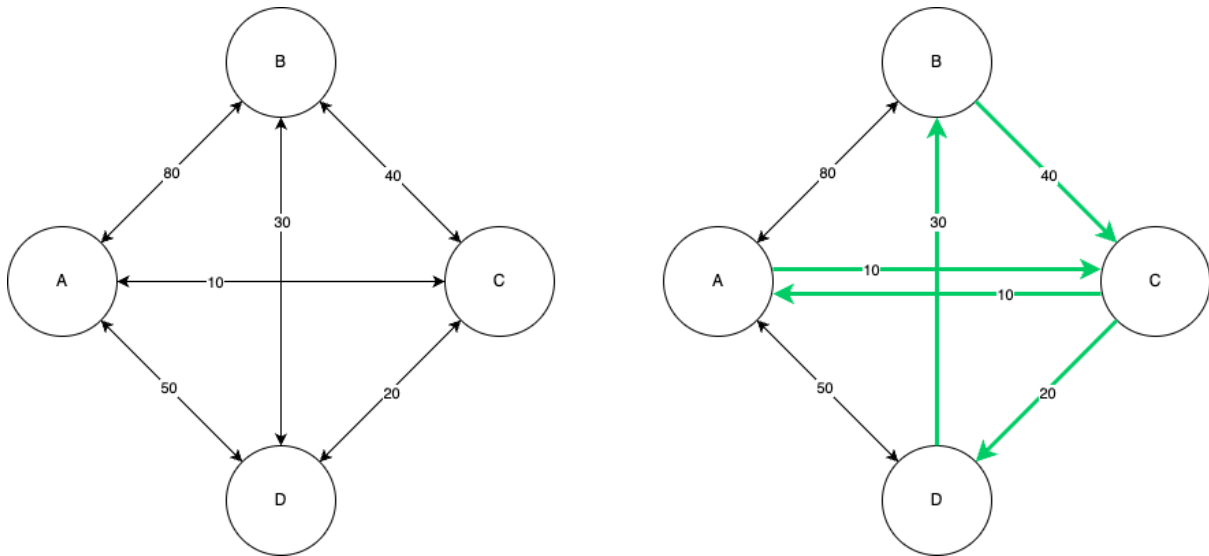


Figure 2.4: Solving a network graph

A different way to model the problem is to display trucks as nodes and use the edges to denote constraints between them. Figure 2.5 shows such a problem: Say that B is a location with emission restrictions that can only be serviced by trucks with a Euro 6 engine. Truck T1 can do this in 80 minutes, truck T2 will take 160 minutes, truck T3 has a Euro 4 engine and therefore does not take part in this calculation.

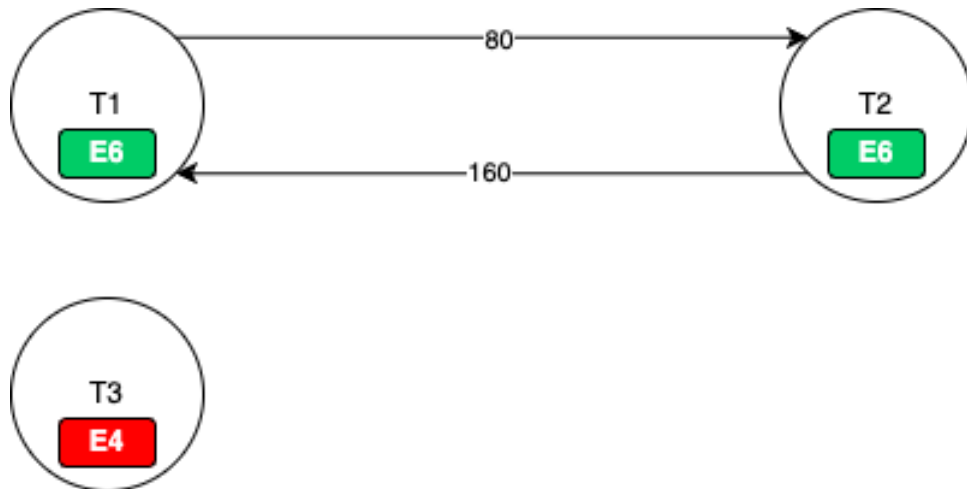


Figure 2.5: Solving a disconnected network graph

This is called a disconnected graph, and that is a great feature because we can solve problems of subgraphs in parallel. A good heuristic is to partition larger graphs in multiple subgraphs with the objective to minimise the weight of the cut.

2.2.2 Centralised, Decentralised, and Distributed Approaches

In this project we will design and compare two algorithms that can efficiently approximate an optimal solution. Several papers [23][24][41][4] present or use a helpful classification of three kinds of algorithmic approaches for solving a network optimisation problem:

1. **Centralisation:** In a centralised approach the algorithm has full knowledge of the problem, and the network can be modelled as a fully connected graph.

2. **Decentralisation:** In a decentralised approach the network is split into multiple parts so that computation is effectively parallelised. This approach remodels the network as a disconnected graph with a null graph at the extreme end.
3. **Distribution:** In a distributed approach information is distributed among nodes who must communicate with each other to find a solution.

The relationship between these approaches is visualised in Figure 2.6.

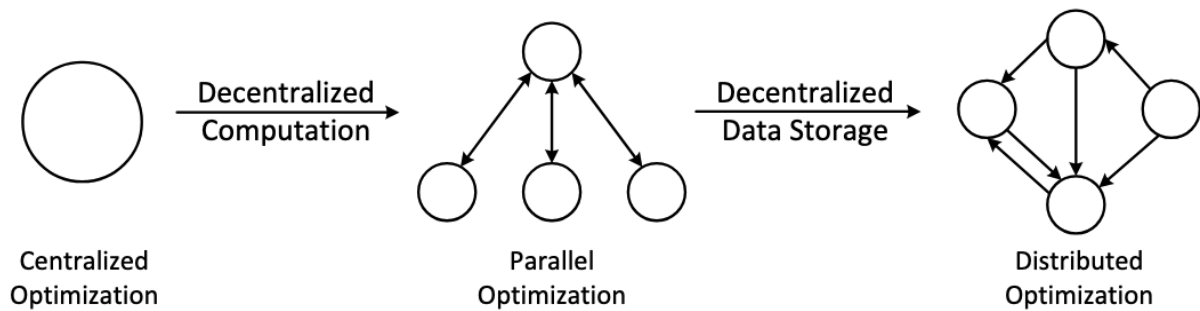


Figure 2.6: Different approaches to network optimisation

Apart from these high-level approaches, there are some well-documented problem definitions with off-the-shelf solvers (such as Google OR-Tools [18]) that are closely related to the problem we aim to solve:

- Task scheduling problem
- Traveling salesman problem (TSP)
- Vehicle routing problem (VRP) with time windows (VRP-TW)
- Capacitated vehicle routing problem (CVRP)
- VRP with pickups and deliveries

The relationship to our problem is easiest explained by mapping them to the most common types of transport in our dataset as displayed in Figure 2.7.

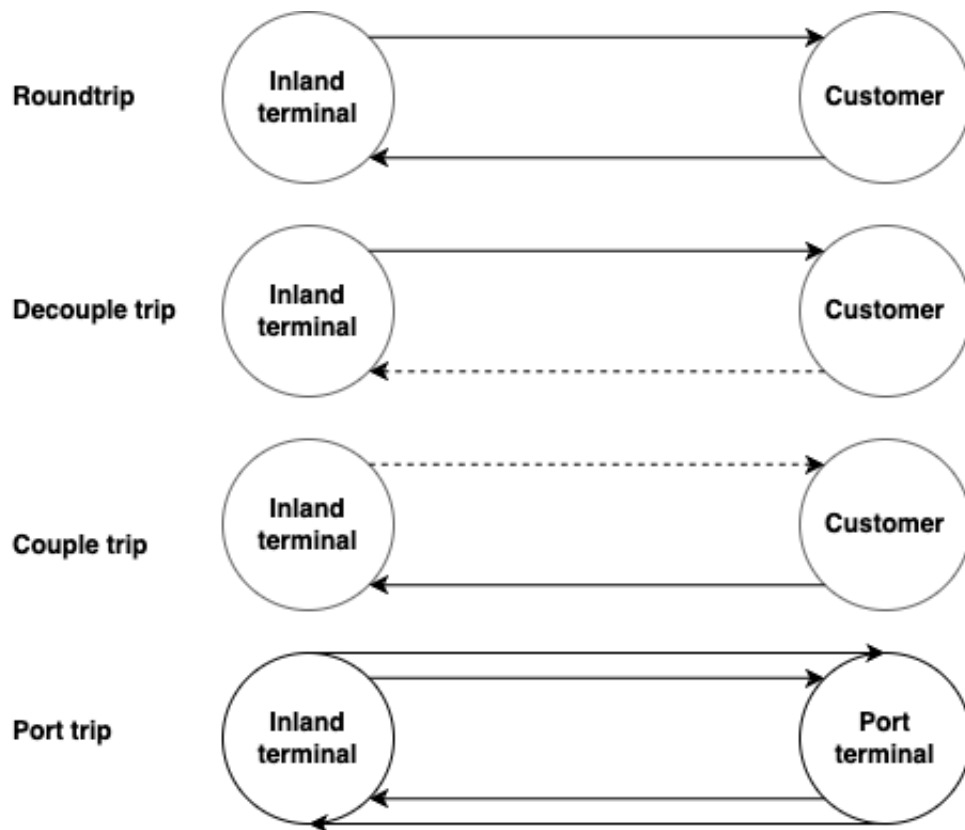


Figure 2.7: Transport types

During a roundtrip a truck drives to a customer and waits for the customer to load or unload the container and then returns to the inland terminal. This problem is closest to what we know in literature as a task scheduling problem. A decouple trip occurs when a truck parks a loaded chassis at the customer and is free to do other work afterwards (a couple trip is its reverse). This problem matches with what we know in operations research as a traveling salesman problem, sometimes referred to as the truck dispatching problem. In this problem, we seek the optimal route to execute a set of deliveries. When a TSP is expanded from a single salesman to a fleet it is called a vehicle routing problem. In our case we have time windows on each customer location, so this would be a VRP with time windows, known as a VRP-TW. Finally, at port terminals containers are loaded and unloaded by crane. Some truck can transport up to three 20 feet containers simultaneously. The problem of choosing the best load combination is known as the container loading problem. The maximum capacity can then be modelled as a capacitated VRP, or CVRP. In addition, containers may be loaded and offloaded at different port terminals, which is known in literature as a VRP with pickups and deliveries.

None of the available solvers for these algorithms can be applied directly to our problem. A common feature of the solvers is that they simplify the problem by modelling a degree of homogeneity in either vehicles, nodes, or locations. This makes sense given the trade-off between optimality and computation time. However, for our project we need to be able to deal with quite a bit of heterogeneity (further discussed in Section 4.1).

Decentral approaches are an alternative way for simplification as the problem is split up in multiple parts. This approach is often more scalable but may lead to suboptimal solutions [1]. We could, for example, split the problem by operational zones of trucks (port, region, local), but we would lose some optimality because port trucks are allowed to execute regional and local rides.

A distributed approach seeks to solve the optimisation problem by distributing both computation and information. Durfee [8] lists five reasons for adopting such an approach:

1. Computation hardware allows for the distribution of asynchronous computation cycles across large numbers of processing units.
2. Many real-world applications are inherently distributed. For example, a set of truck drivers that work together in a fleet may be spatially distributed, the tasks they need to perform may be temporally distributed, and there may be functional distribution between their individual skillsets.
3. Structuring a problem into simpler isolated pieces creates a system that will be easier to build, debug and maintain. In computer science this is known as a microservices pattern.
4. Coordination and cooperation are complex phenomenon that are an interesting field of academic study because they offer an intersection between computer science and sociological, managerial, and organisational theory.
5. Computer systems must interact with humans to be accepted in society and business processes. As a result, a system should be flexible and be able to coordinate and cooperate with humans and itself.

2.2.3 Greedy Search versus Multi-Agent

Two mainstream approaches to solving a network problem are variants of greedy search and multi-agent systems. We will list some advantages of both approaches.

The main advantages of a centralised greedy approach are:

1. **Simplicity:** It is easier to design and implement, because there is only a single entity making the decisions [44].
2. **Performance:** Early assignment by greedy search improves performance, because the problem size will be reduced at each step [36][27].
3. **Deterministic results:** Each input will generate the same output. Our system is intended to run in production, predictability is a benefit that will generate trust among users. In multi-agent systems conflicting strategies of independently acting agents may lead to variability.
4. **Local optimality:** A greedy approach uses heuristics to generate a solution fast. In logistics heuristics like "shortest path" have been proven to be highly effective in approaching the global optimum [45][29].
5. **Easier to debug and maintain:** It is generally easier to understand the logic, trace errors, fix problems and add new functionality. In a multi-agent approach a strange outcome will be the result of multiple agents interacting with each other.

The main advantages of a multi-agent approach are:

1. **Complexity:** Multi-Agent systems excel in their capabilities for managing complex logistics operations through distributed decision-making [25].

2. **Distribution:** The structure of the problem is inherently distributed: spatially, temporally, and functionally. A multi-agent approach might be better suited model the real environment [7].
3. **Human-machine interaction:** A multi-agent approach makes it easier for humans to understand and interact with the software. A user may interact with a single agent, rather than with the full problem. Cooperation and coordination with humans are essential to solve the problem, and a multi-agent solution may be a better fit with the organisation [13].
4. **Dynamic environment:** Multi-Agent systems show potential for enhancing adaptability and resilience in supply chains [12]. Updates (for example about truck positions) come in continuously. It is easier to calculate the effect of an update in the agent responsible for the transport than to recalculate a full solution at every change in context.

2.2.4 Large Language Models

The recent surge in agents based on large language models (LLMs) is worth mentioning as it opens up a whole new kind of algorithm for automation and decision support in logistics. These systems can reason over complex multi-modal data and interact with human planners in natural language. This has the potential to bridge the gap between algorithmic optimisation and human expertise [26]. This is still an emerging field which may need some more development before applying LLM-based solutions in operational systems. Prior research in solving a trucking problem with reinforcement learning also revealed that [5] explaining to a human why an algorithm based on neural networks made a specific decision can be quite challenging.

2.3 Conclusion

In summary, the optimisation of containerised road transport planning reflects challenges faced in global logistics: increasing operational complexity, the need for resilience in the face of disruptions, and the drive towards automation and digitisation. While this dissertation is scoped to the context of a specific inland terminal network in the Netherlands, the issues addressed are present anywhere in logistics: balancing capacity, regulatory compliance, dynamic disruptions, and efficient assignment. Insights and solutions developed here should be applicable in a broader context.

A review of the literature reveals several algorithmic approaches, from classical heuristics and centralised optimisation to distributed multi-agent systems and agentic systems powered by large language models. Each approach offers distinct advantages and trade-offs in terms of scalability, explainability, adaptability, and human-machine interaction. While LLM-based agents hold great promise for the future, especially in bridging the gap between algorithmic optimisation and human expertise, their operational deployment in safety critical, real time logistics remains an open challenge.

This dissertation therefore focuses on the more traditional methods. These methods offer higher potential with regards to computational tractability, transparency, and practical applicability. The following chapters will translate these insights into concrete requirements, system design, a prototype implementation, and the empirical evaluation of two algorithms.

3 Project

In this chapter we will set the scope of the project, define our project goals, list the high-level requirements, and discuss ethical consideration, project management, planning, and risk.

3.1 Scope

The scope of this project is deliberately limited to the assignment of drivers to regional transports by truck within a real-world inland terminal network.

By narrowing our focus, we are able to achieve three key objectives. First, we address a problem of high practical relevance where insights and learnings can be generalised to other contexts, as the core challenges of assignment, capacity balancing and compliance recur throughout the logistics sector. Second, we can develop algorithms in an area where data is available, constraints are well-understood, and results can be meaningfully evaluated by contrasting them with human generated transport plans. Third, we ensure that we have a manageable workload and can deliver something meaningful that actually helps human planners in their daily work.

For completeness, we explicitly limit our scope by excluding the following parts of the optimisation problem:

- **Excluding barge and train planning:** These are separate optimisation problems in their own right with distinct constraints and decisions. Together with trucking these problems are part of the larger network optimisation problem. We acknowledge the interaction effects between these modalities, but we will treat them as exogenous to our problem.
- **Excluding demand forecasting:** There is an ongoing collaboration with the University of Twente to develop a demand forecasting model. The goal is to organise sufficient transport capacity two weeks in advance. For this project, however, we assume that the set of drivers, trucks, and transports is fixed and known, allowing us to focus on the assignment problem.
- **Partially excluding port transports:** Port trucking introduces additional constraints (e.g., multi-day time windows, multi-container loads) that differ from regional transports. The developed system should allow for manual assignment of port transports, but algorithm development is restricted to regional transports. A group of undergraduate Oxford students is currently working on a port trucking algorithm as part of their Group Design Project, this is showing some promising results.
- **Excluding real-time monitoring:** This would lead to extra requirements on the graphical interface and the required data is currently unavailable. However, the algorithms developed are designed to support replanning to some extent.

We will exclude the use of LLMs as discussed in Section 2.3 and focus on two algorithmic approaches: A centralised greedy approach, and a decentralised multi-agent approach. LLMs offer high potential for human-machine interaction, but are likely not suitable for application in operational environments for reasons of transparency, explainability, and predictability.

This scoping ensures that the project remains feasible and practical as we aim for real-world application. By concentrating on a well-defined, high-impact subproblem, we lay the groundwork for future extensions to more complex, multi-modal, and dynamic planning environments.

3.2 Goals

We will set our ultimate goal to:

“The design of a system that enhances the capabilities of human transport planners by reducing cognitive load, increasing operational efficiency, and improving decision-making quality.”

In more words we intend to design a software application that is able to generate planning proposals that are both technically sound and practically useful. Human planners should be able to interact and override decisions giving the dynamic nature of environment and the impossibility of covering all exceptions.

To achieve this we adopt three explicit goals:

- G1** Capture the problem in a fitting data model.
- G2** Design and implement an appropriate software architecture.
- G3** Explore and evaluate algorithmic solutions for automating planning proposals.

3.3 Requirements

The requirements for this project are derived from a combination of practical experience with transport planners, discussions with industry partners, and a review of the literature on logistics planning systems. They are designed to ensure that the software is both usable in real life and capable of addressing the core challenges identified in the background and scope sections.

We have grouped the requirements into two categories: Functional requirements and non-functional requirements. Functional requirements capture the core planning capabilities, what planners need to be successful in their daily work (see Section 2.2). Non-functional requirements are derived from the nature of a planner’s work and his work environment (see Subsection 1.1.3), they capture requirements that make the system usable.

3.3.1 Functional Requirements

FR1 Planners should be able to manage cargo and capacity constraints.

Justification: Effective planning requires the ability to match available capacity (drivers, trucks) to transport demand, while taking constraints into account (for constraints, see Section 4.1).

FR2 Planners should gain insight whether transport capacity is meeting demand.

Justification: Planners need visibility into whether the available resources are sufficient to meet transport requirements. This supports proactive decision-making and helps avoid last-minute shortages or overcapacity.

FR3 Planners should be able to manually assign trips to drivers.

Justification: While automation is a goal, planners must retain the ability to override or adjust assignments based on operational knowledge, exceptions, or customer requests.

FR4 Planners should be signalled when unwanted situations could occur.

Justification: Early warnings about potential issues (such as capacity shortfalls, regulatory violations, missed delivery windows) enable planners to take corrective action before problems escalate.

FR5 Planners should be able to understand why an algorithm proposes a specific solution.

Justification: Explainability is critical for user acceptance, especially in high-stakes environments. Planners need to trust and verify algorithmic recommendations, and be able to communicate reasoning to stakeholders.

3.3.2 Non-Functional Requirements

NFR1 Multiple planners need to be able to work simultaneously.

Justification: In practice, planning is a collaborative activity, with several planners working together. The system must support concurrent access and updates without data loss or conflicts.

NFR2 Initial system load should be under 1 minute, interactions should be under 10 seconds.

Justification: Planners operate under significant time pressure (see Subsection 1.1.3). Slow interfaces can lead to frustration, errors, and reduced productivity. 10-12 seconds is a common upper bound for user interactions [21][30]. We found no research on upper bounds for initial application loads, the ERP-system planner currently use takes around 1 minute to load which is why we adopted that value.

NFR3 Synchronisation with ERP systems should be near instant.

Justification: Accurate, up-to-date information is essential for effective planning. Delays in data synchronisation can result in missed opportunities or operational errors.

NFR4 Algorithmic planning proposals should be generated in under 2 minutes.

Justification: Automated planning must be fast enough to support real-time or near-real-time decision-making [3]. A study that comes close to what we intend to do generates near-optimal solutions within a few minutes for problem instances involving 50 customers [46]. Others, however, found high-performance solutions with runs in the range of 1 to 10 seconds [38]. We will adopt a 2-minute limit to balance computational complexity with operational needs.

3.4 Ethical Considerations

To be able to benchmark our algorithmic solutions and to stay as close to reality as possible we intend to use real-life data in this dissertation. That raises ethical concerns as this data includes personally identifiable information of drivers and planners. The raw data includes names, phone, numbers, and email addresses of drivers and planners. It also includes driver

work schedules and license plates of their assigned trucks. Furthermore, the timestamps of assigned transports reveal actual location whereabouts. Finally, the data also includes the names and addresses and the number transports to customer locations, which is commercially sensitive data.

3.4.1 Ethical Approval Process

Using this data for research is secondary usage and requires consent from the owners of the data being the people represented by it and Van Berkel Logistics where it concerns commercially sensitive data. We went through an ethics approval process with the Ethics Committee of Oxford's Computer Science Department where we explained the intended use of the data. We proposed to only collect anonymised data regarding, trucks, drivers, chassis, locations and transports for several planning days.

To safeguard the privacy of drivers and to avoid leaking commercially sensitive data, the following anonymisation measures were proposed and taken:

- Replacing driver names with random names;
- Deleting driver email addresses;
- Replacing truck license plates by random characters;
- Replacing chassis license plates by random characters;
- Deleting location names, addresses, phone numbers, email addresses and websites;
- Replacing location code identifiers by random characters when this location is a customer (codes of container terminals will remain as is as they are not commercially sensitive);
- Shifting transport dates to a random date (e.g., data collected on May 15, 2024, will have timestamps shifted by an arbitrary number of days).

Van Berkel Logistics signed a letter of consent (see Appendix Section A.1) and the ethics committee concluded that no ethical approval nor personal consent from all drivers was needed provided that the anonymisation measures were taken.

3.4.2 Remaining concerns

There are a number of remaining concerns that should be raised when the application developed in this dissertation is used in real life:

- **Privacy and data protection:** The database will contain personally identifiable information. We should ensure that people are aware that their data is used and stored.
- **Data security:** We should have processes in place that prevent unauthorised access, remove old data, and reduce any risk of data leakage.
- **Surveillance:** Monitoring drivers or planners can lead to feelings of mistrust and may reduce job satisfaction. We should be careful with monitoring driver and planner behaviour and exposing this data to managers.

- **Data usage transparency:** Drivers may not understand how decisions are made, we should inform them whenever their data, specifically with regards to personal preferences, is updated by a planner.
- **Fairness:** Bias in algorithms may create inequality between drivers, we should avoid this, and be transparent about it if it does occur.

3.5 Project Management and Planning

Project Management

The project proposal, submitted in 2022, included a diagram with all the steps needed to complete this project (see Appendix Section A.3). These grouped these steps in five phases:

1. **Design phase:** Gathering system requirements, and designing new planning processes, an IT architecture, a data model, and possibly a set of GUI web designs.
2. **Research phase:** Study options for tooling and techniques and do a lot of thinking on how to develop, test, and integrate all software components. The multi-agent algorithm is a major task in this phase.
3. **Development phase:** Workout and build the multi-agent architecture. If scope is extended, build the other pieces, and integrate all software components.
4. **Simulation phase:** Test the software, run through various scenarios, and do a couple of test runs with Van Berkel Logistics to obtain feedback.
5. **Implementation phase (initially out of scope):** Write user documentation and plan the implementation. Do iterative development based on user feedback

Looking back, all of the steps have been taken and there currently is a working version of Squid used in the daily operation of Van Berkel. In that regard, my project management was outstanding.

Planning

However, I did not complete these tasks within the estimated 1600 hours listed in the proposal. I have not been keeping track of time, but this has been my full-time job for the last 2,5 years. I could have submitted my dissertation earlier, there was a minimum viable product available in 2023, but instead I requested a single term extension followed by a double term suspension. There were two reasons for this:

1. I had launched Squid and became dependent on it for my income. This meant that operational issues and customer requests took priority over working on my dissertation.
2. I became a father twice. It is amazing to have kids, but it makes it harder to balance work with family life. It reduced my free time in evenings and weekends and dropped my dissertation work a further position down the priority list.

I could have been more strict on the planning and probably would have been a good idea to adopt a more limited scope for this dissertation. On the upside, I am quite proud where we stand today and do not regret putting in much more hours than initially planned.

Risk Management

My project proposal listed a number of risks. We have mitigated these over the course of the project:

- **Data availability:** Squid was designed to enrich the existing data (for example by finding the GPS-coordinates of each location) and to retrieve information from the heads of planners. In a way the software itself took care of ensuring data availability.
- **Data quality:** Data quality has been improved since planners started using the software. It is easy to populate fields once, but they will only be kept up to date if the data is used in an operational process.
- **Van Berkel Logistics pulling out of the project:** Luckily they have not, they still are supportive of Squid and we have since signed a second customer.
- **Integration with external systems:** This has slowed us down enormously. The integration still has room for improvement, a push-integration would be a major improvement over the current pull-integration, but we succeeded in setting up the required data connections.
- **Willingness of planners:** This was one of my major worries. Planners, however, appeared very cooperative and they were and still are very enthusiastic about using the software in their daily work.

4 Design

It is tempting to focus solely on a planning algorithm, but more components are required before a transport planner can put an algorithm to practical use. This chapter describes the design and architecture of the developed transport planning software. In the Background chapter 2 we described the general process the software needs to facilitate. The appendix includes the more detailed original process diagram (A.4) and user story A.5, both written in 2022. To summarise, a planner needs three features to be present for a minimum viable product:

1. Management of driver availability
2. Assigning transports to drivers (manual or automatic)
3. Communication with drivers or ERP-system to send out the plan

The design of our system should facilitate this, but in order to generate a planning or even to visualise the correct information, we need map out the constraints, capture all information in a data model, design an IT architecture that includes all required components, and figure out how to integrate everything. We will list both the capacity and demand constraints per entity below.

4.1 Constraints

There are several constraints on both the transport capacity and the transport demand. On the capacity side we have constraints affecting drivers, trucks, and chassis. On the demand side the constraints are related to the handling of the cargo.

4.1.1 Capacity Constraints

Driver constraints:

- **Working hours:** Typical agreement from 05:00 to 17:00, Monday to Friday, but this varies per driver.
- **Overtime:** Acceptable amounts vary per driver, but usually between 30 and 60 minutes.
- **Legal restrictions:** Maximum of 4,5 hours of uninterrupted driving, followed by a 45-minute break [37].
- **Certifications:** Varies by truck and cargo, hazmat goods are an example.
- **Preferences:** Home sleeping, long-distance driving, and task types.

Truck constraints:

- **On board units:** Required in Belgium and Germany.

- **Leading axle:** Affects maximum weight capacity.
- **Sleeping cabin:** Enables overnight stays.
- **Euronorm:** Determines environmental access restrictions.

Chassis constraints:

- **Axles:** Affects maximum allowed container weight.
- **Type:** Determines which container types are supported.

4.1.2 Demand Constraints

Demand constraints relate to the handling of the cargo:

- **Delivery datetime:** A container usually has an agreed upon loading or discharge time at a customer location. It depends on the customer how much deviation around that time is acceptable. Sometimes the handling location does not have a specific time but a specified time window.
- **Driving duration:** The duration it will take to drive to the cargo handling location (and return).
- **Handling duration:** The handling duration depends on the service type. Sometimes a container may be decoupled at a customer location, sometimes it is unloaded by a crane, sometimes a driver will have to wait until the cargo is offloaded from the container. The service types are displayed in Figure 4.1.
- **Service type:** Figure 4.1 also shows that there is an interdependency between transports. The service type determines what the driver can do next. It should also be noted that when a driver returns from a location with a container, the cargo and the owner of the container has an effect on what can be done next. For example, after live loading a container we can either unload that container at the inland terminal or the sea terminal of departure (provided that the drop-off window is open). After live unloading a container, we can unload the container at a terminal, decouple it, or load it at another customer provided that that customer requires the same container type of the same shipping line.
- **Pre-transport:** If the pre-transport of the trip has not been completed, it will not be possible to transport the container. A container might still be on board a ship.
- **Other restrictions:** A container can be on hold, because of customs inspection, and some containers require gas measurement.

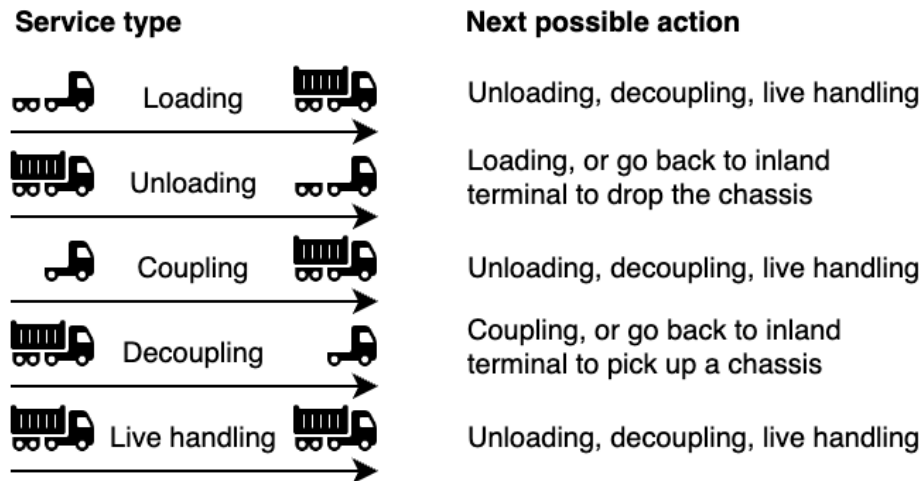


Figure 4.1: Possible chassis states at transport origin and destination

4.2 Data Model

We need to be able to represent the constraints, the assets (trucks, drivers, chassis), transports (and related locations), some system-level data (like users) and how all this relates together. Roughly speaking the data can be grouped in four groups:

1. **System-level data:** Data that we need for the system to operate. Currently this is just a table for users that we need for authentication. We will likely add additional system-level tables in the future like settings, migrations, and logs.
2. **Master data:** Data that describes our transport capacity. This includes tables for drivers, trucks, chassis, and locations.
3. **Transport data:** Data that describes our transport demand. This includes tables for containers, transports, and stops.
4. **Planning data:** Data that reflects the decision making of a planner. Which driver will execute which transport is stored in the assignments table.

Each entity, the attributes per entity and the relationships between entities are represented in the entity relationship diagram (ERP) shown in Figure 4.2. Some observations that can be made are that the data is very relational, assignments tie everything together, locations play a role in almost every table and timestamps are important, most of the attributes of a stop are time related.

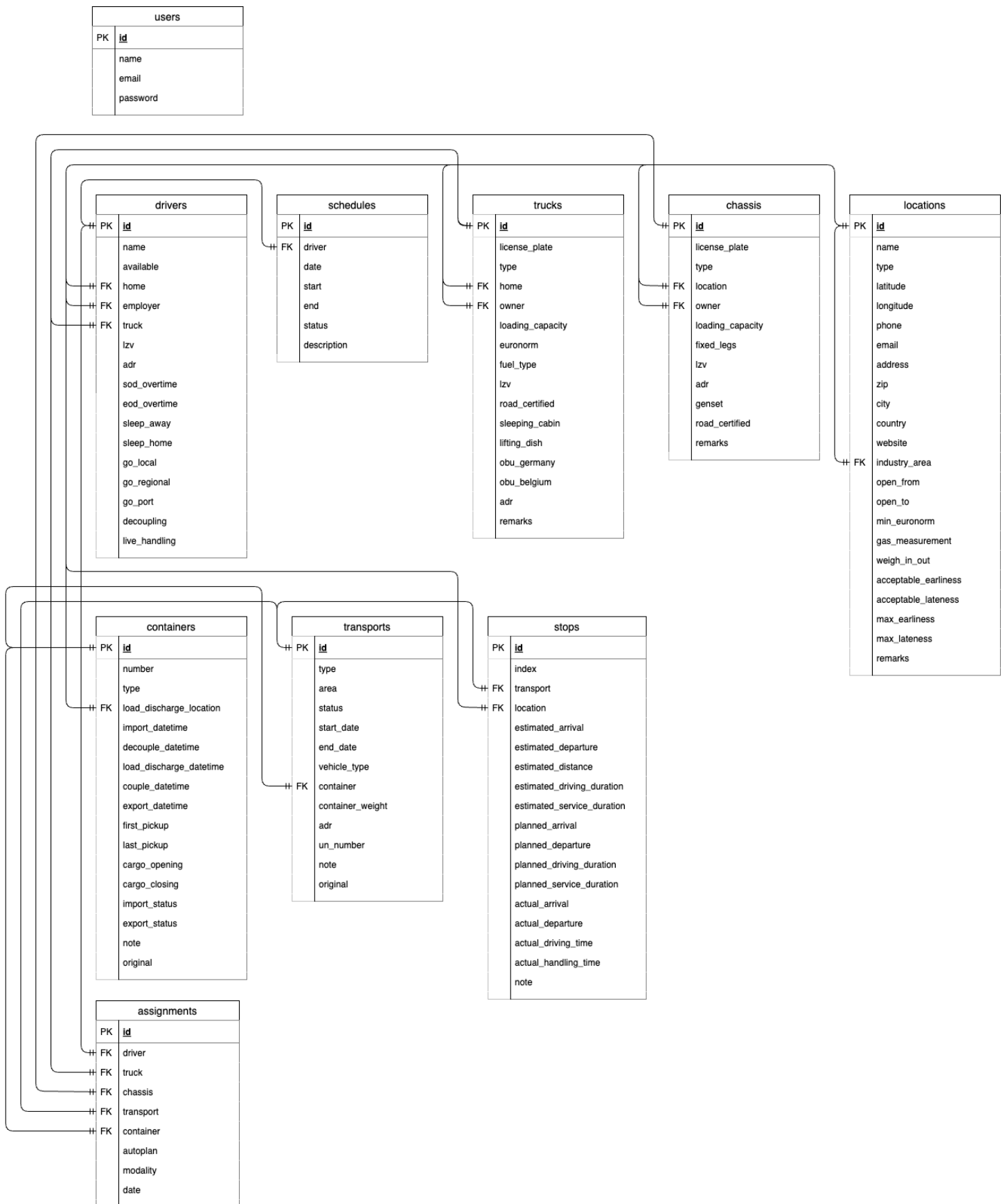


Figure 4.2: Entity relationship diagram

4.3 Architecture

Figure 4.3 contains an architecture diagram of all the software components that make our container transport planning system available to its users. We will discuss most components in more detail in Chapter 5. We deliberately chose for a microservices architecture to keep components maintainable and to be able expose components like the OSRM API and Geocoder API to third parties. Tenant databases are separated by customer to avoid any risk of data leakage.

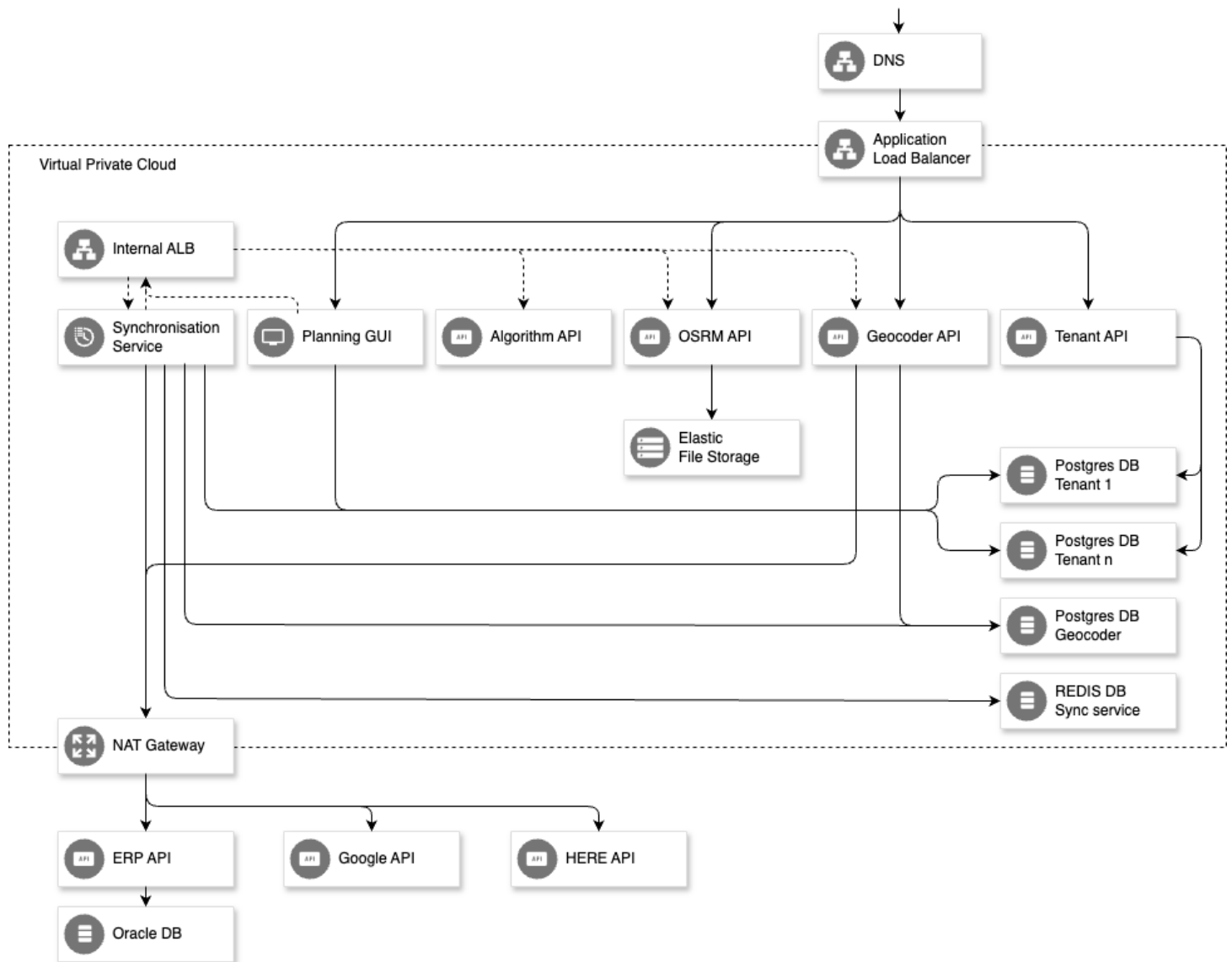


Figure 4.3: Architecture diagram

4.4 Integration

The sequence diagram in Figure 4.4 displays how information flows through the software components. First, we need to ensure that we show the correct master and transport data. The synchronisation service is responsible for this process. Whenever a request comes in by cron-job or API call, the synchronisation service will call the desired ERP API to request all relevant data from the ERP-system. For each returned object, the respective data is put in a queue in a REDIS database. The queuing mechanism of the synchronisation service has several worker

threads that process the raw data. The data format of ERP systems differs, therefore there are per-tenant processing adapters. Sometimes additional data is required, such as GPS-coordinates for locations. When the worker completes a task it will create, update, or delete the processed record from the database. If it fails to complete the task, the failed records will be in a failure queue for manual processing. There are triggers configured to send out notifications on create, update, and delete actions in Postgres.

Second, one of our requirements was that multiple planners need to be able to work simultaneously in the same application. The sequence diagram below and the steps above show how this problem is now solved: Once planner A assigns a transport this will result in a database change, Postgres will fire off a notification which is caught by the server-side part of SvelteKit and transmitted over websockets to the client-side application that instantly updates the screen of planner B.

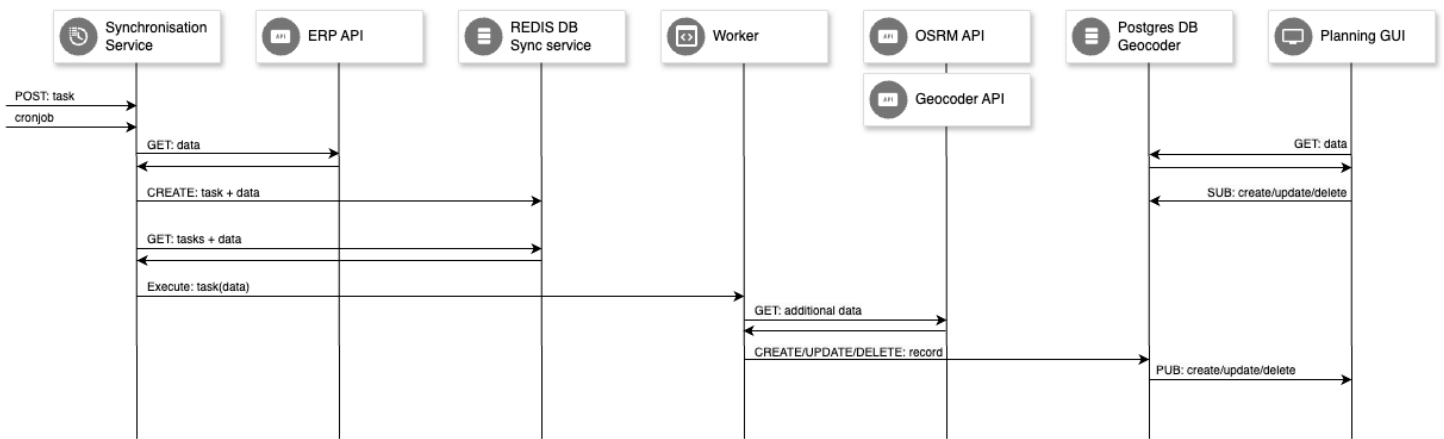


Figure 4.4: Sequence diagram

4.5 Software Components

Now that we have designed a data model, a software architecture, and all communication interfaces, we can discuss the design of several software components.

Graphical User Interface

The planning GUI is the primary interface used by planners to manage master data, assign transports to drivers, and monitor ongoing operations. Planners often work in high-stakes, time-sensitive environments (see Section 1.1.3), so we set the intention to designed an interface that reduces cognitive load by minimising the displayed information to help users focus and make the right decisions. In addition, we limited colour usage to warnings and alerts to help identify issues faster.

We chose to build this GUI in SvelteKit [40], a modern web framework that balances performance and developer productivity. This decision was driven by both personal and technical factors. On the personal level, the author has extensive experience with frontend frameworks such as Ember, React, React Native, and Vue. Familiarity with these tools accelerates development by offloading low-level tasks to the framework, while vibrant ecosystems provide access to robust libraries (e.g., Chart.js was used to draw Figure 2.3) and community support. At the same time, the author values the simplicity and performance of Vanilla JS. Writing pure JavaScript avoids many of the pitfalls of large frameworks: version churn, package bloat, and framework fatigue.

SvelteKit offers a compelling middle ground. The project at hand was complex enough that using only Vanilla JS would have been too time-consuming. SvelteKit, even while still in beta in 2022, brought first-class support for features that other frameworks were retrofitting, such as pre-rendering, server-side rendering, hydration, and a strong focus on form actions. Its ability to unify backend and frontend logic in a single codebase proved invaluable.

The 2022 State of JavaScript survey [19] highlighted challenges in frontend framework retention, with very a very low retention rate for Svelte:

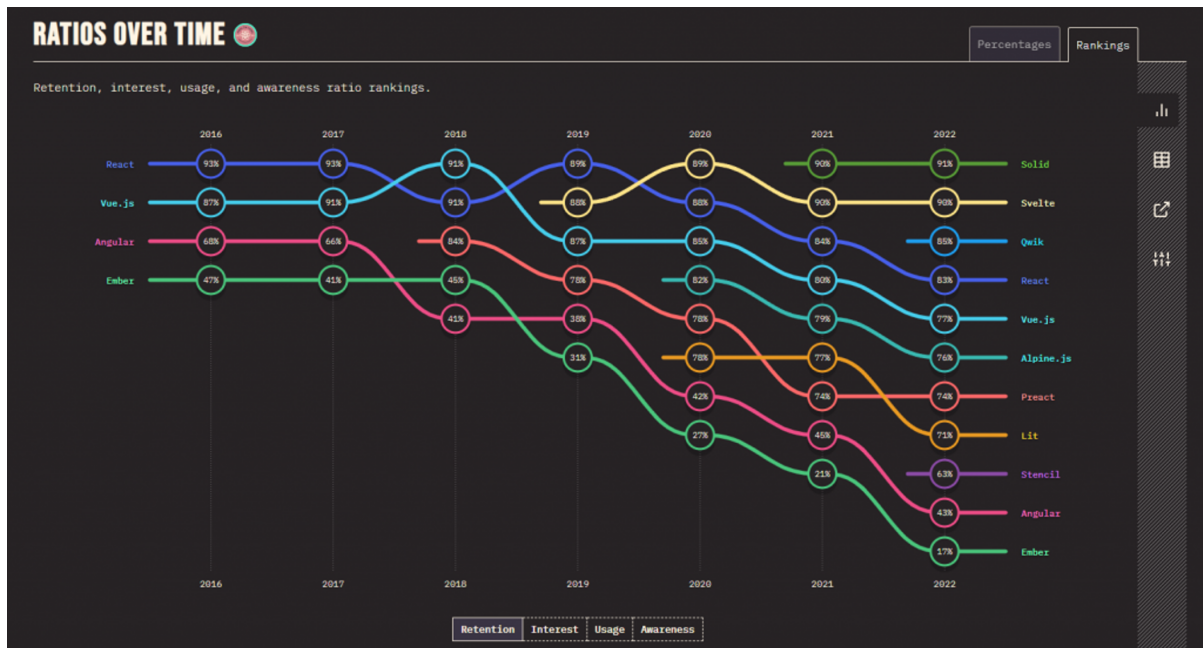


Figure 4.5: 2022 retention ranking of frontend frameworks

Despite the risks, SvelteKit's beta status and the inherent learning curve of any new tool, the choice has proven successful. Today, SvelteKit is out of beta, stable, well-supported, and backed by an active community. It has been a joy to code all the needed components.

Synchronisation Service

The synchronisation service' main task is retrieving and processing data from external ERP-systems. It exists so that the data shown in the planning software matches with what resides in the ERP-system. The communication sequence is explained in Figure 4.4. This service should consist of three components:

1. An API, so requests for data can be triggered on-demand;
2. A built-in cron job handler that checks for master data updates around midnight and for transport updates every minute;
3. A built-in queuing mechanism that executes tasks from queues.

We decided to write this service from scratch as the processing of data requires in depth knowledge of what specific fields mean. Special care was taken to separate out adapters of multiple customers to be able to handle data from different ERP-systems. Another requirement that came up during development is the need to separate fast and slow processing. Transport updates of transports planned for today or tomorrow should be processed fast, where a new booking a month from now has a much lower priority.

Planning Algorithm

We decided to write two planning algorithms (see Section 3.1) which we will compare with each other and a planning solution generated by human planners. All three methods should adhere to the same constraints and aim to optimise the same goal. Given the constraints defined above (see Section 4.1) and the analysis of our optimisation problem (see Section 2.2) we can define the following objective function:

$$\textbf{Objective: } U = \alpha \cdot H_c + \beta \cdot U_t + \gamma \cdot H_d$$

Subject to hard constraints:

- $C_1 \leq$ working hours limit
- $C_2 \leq$ mandatory driving and rest times
- $C_3 \leq$ driver certifications
- $C_4 \leq$ truck and chassis limitations
- $C_5 \leq$ cargo constraints

Subject to soft constraints:

(4.1)

- $S_1 =$ driver preferences
- $S_2 =$ acceptable deviations in delivery time

Where:

- $U =$ overall utility of driver assignments
- $H_c =$ customer happiness
- $U_t =$ transport utilisation
- $H_d =$ driver happiness
- $\alpha, \beta, \gamma =$ weight coefficients balancing H_c, U_t, H_d

Our objective function aims to maximise the overall utility of truck-driver combinations by optimising the allocation of transport capacity to meet the demand for transportation. The objective function must operate within a set of hard constraints and soft constraints, where the latter may be partially violated to achieve the overall optimisation goals. We will include the following constraints in our algorithms:

- **Driver base location:** Driver should start and end at their base location.
- **Driver working hours including overtime:** Drivers should not work beyond their working hours plus acceptable overtime limits.
- **Driver hazmat certification:** Non-hazmat should not transport hazmat goods.
- **Start and end locations:** Transports should start at their start location and end at their end location.
- **Delivery time window:** Transports should reach their delivery location within the maximum defined deviation from the delivery time.

- **Transport duration:** The duration of a transport is equal to the driving duration and the handling duration. When a transport is assigned to a driver the driver will be unavailable for other work for the duration of the transport.

That does mean that we are also excluding several constraints:

- **Driving time restrictions**

Justification: We scoped our problem to regional work. These are typically short drives under two hours. When a container is loaded or unloaded at a dock the waiting time for the driver counts as rest. As a result, drivers usually do not exceed the legal limits of driving time when doing regional work.

- **Driver preferences**

Justification: Driving preferences for long distance work or sleeping away from home are not relevant for our problem as we will only include the drivers that prefer to work short distance and sleep at home.

- **Truck on board units**

Justification: Regional work typically does not involve crossing borders, on board units are not relevant.

- **Truck leading axle**

Justification: When doing regional work drivers only transport single containers. The truck/chassis combination is unlikely to be unsuitable for container weight. Leading axles start playing a role in port trucking.

- **Truck sleeping cabin**

Justification: Not relevant for our problem, drivers will sleep at home.

- **Truck euronorm**

Justification: Not relevant for our problem, customer locations for container delivery are not in city centers. The only locations with a euronorm restriction are the port terminals in Rotterdam.

- **Chassis axles**

Justification: Chassis axles are relevant with regards to the weight a truck may transport. Van Berkel Logistics, however, uses standard 3-axle chassis that can carry the maximum amount of allowed weight.

- **Chassis type**

Justification: There are chassis that only can only transported certain types of containers. Van Berkel Logistics, however, uses standard chassis that can transport any type.

- **Pre-transports**

Justification: For sake simplicity we are assuming that all transports are available. This is also a constraint that should be part of the pre-processing and not of the algorithm itself.

- **Other transport restrictions**

Justification: Same as pre-transports, cargo that is not available should not be input for our algorithms.

Lastly, we should discuss three simplifications of reality that we adopted for ease of calculation. First, we are transforming transports to homogenous service types. The various service types and their effect on chassis state are displayed Figure 4.1). Loading and unloading does not happen for regional work, so we can ignore those. Decoupling typically occurs at customers that handle multiple daily containers. What usually happens is that a driver decouples a chassis and immediately couples a finished container. For sake of simplicity we will convert decoupling transports in live handling and exclude couple transports from the dataset. This gives us a homogenous set of transports that go from A to B to A.

Secondly, we will assume symmetric driving duration, which means assuming that the time it takes to drive to a location is equal to the time needed to return. This is not always true in real life, but it allows us to calculate with a single duration value.

Thirdly, we will consider merged transports as an input. Figure 4.6 shows an example of what the planners call "throughloading". This looks like a combination of two transports, but it actually is a combination of three transports: The driver decouples a full container, couples an empty container, drives to different customer and waits for container to be loaded, and then returns to the terminal. This is a smart optimisation trick that can only be done if the delivery times match up, if customers are located relatively close to each other, if container types match up, and if the shipping lines that own the containers match up and allow such arrangements. We treat these combinations as single transports, because we lack the data on shipping lines and there is an argument to make for finding such cases in a preprocessing step.

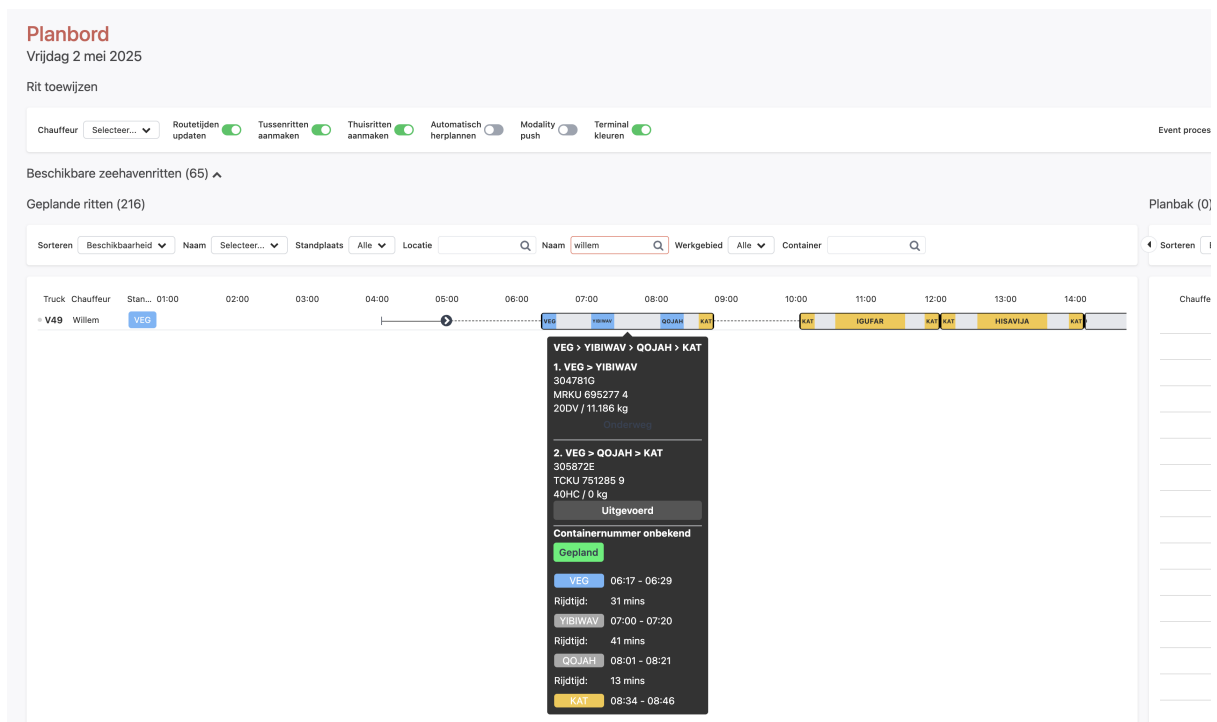


Figure 4.6: Throughloading Example

5 Implementation

In Chapter 1, we set out to address a fundamental challenge in containerised road transportation: The need to support human planners in an increasingly complex and dynamic environment. We promised to develop a system that would reduce cognitive load, increase operational efficiency, and improve decision-making quality, while maintaining the flexibility needed in real-world operations.

To deliver on this promise, we set three explicit goals: capturing the problem in a fitting data model (G1), designing an appropriate software architecture (G2), and exploring algorithmic solutions for planning automation (G3). These goals were further refined into specific functional requirements from managing constraints (FR1) to explaining algorithmic decisions (FR5) and non-functional requirements covering aspects like system responsiveness (NFR2) and multi-user support (NFR1), see Chapter 3.

This chapter describes the implementation of a prototype system that aims to satisfy these requirements while adhering to our scope limitations (see Section 3.1. The prototype consists of several key components:

- A web-based graphical user interface for planner interaction
- A PostgreSQL database for maintaining the system’s data model
- A synchronization service for near real-time data integration
- Supporting services for geocoding and route calculation
- Two distinct algorithmic approaches: a centralised greedy search and a multi-agent system

The implementation of each component involved careful consideration of various technical options and trade-offs, which we will discuss in detail. More importantly, this prototype serves as a vehicle for experimental validation - allowing us to evaluate how well our chosen approaches meet both the technical requirements and the real-world needs of transport planners.

The following sections detail the implementation of each component, the rationale behind key technical decisions, and how these choices contribute to meeting our goals and requirements. We conclude the chapter with a critical assessment of the prototype’s strengths and limitations, setting the stage for the evaluation presented in Chapter 6.

5.1 Graphical User Interface

Building the Graphical User Interface (GUI) has been an iterative process where we moved from wireframes (see Appendix A.6), to designs (see Appendix A.7) to an actual web application (see Appendix A.8). There were five notable technical challenges. First, server-side rendering proved not very useful because of the dynamic nature of our data. In SvelteKit you

should also be very careful about data leakage when dynamic data is rendered server-side because stores are shared [28].

Second, our data model is comprehensive, but the queries to load the relevant data for a single day became quite complex. That data had to be transferred, injected in multiple client-side stores, and rendered into visualisations. Getting initial application load under 1 minute proved a challenge, but we succeeded by adding indexes to our tables in database, transferring the data in Protobuf and optimising our client-side renderings with virtual lists.

Third, to keep the actions of multiple planners in sync and the data in sync with the ERP-system without needing a browser refresh we implemented websockets. SvelteKit does not support websockets, so this was quite a complex thing to implement and we had to write a custom Socket.io integration for SvelteKit. In addition, we found that we really had to limit the amount of traffic to the browser, because many updates happen in the ERP-system. We succeeded in this by making channels for each date and only subscribing to the relevant date.

Fourth, when we signed a second customer we ran into the problem of multi-tenancy. We didn't want to build two applications, but putting a new feature live for one customer creates a problem for two customers if the feature contains a bug. We have since made our environment single-tenant so customers can temporarily use different versions of the GUI.

Finally, the original ERP is much like a giant spreadsheet. The design philosophy of our GUI was to omit as much data and colour as possible, so a planner only sees what he has to see at the right moment in time. We partially succeeded in this, but a recent survey showed that planners are still switching back to the ERP to find additional data, so there is room to improve!

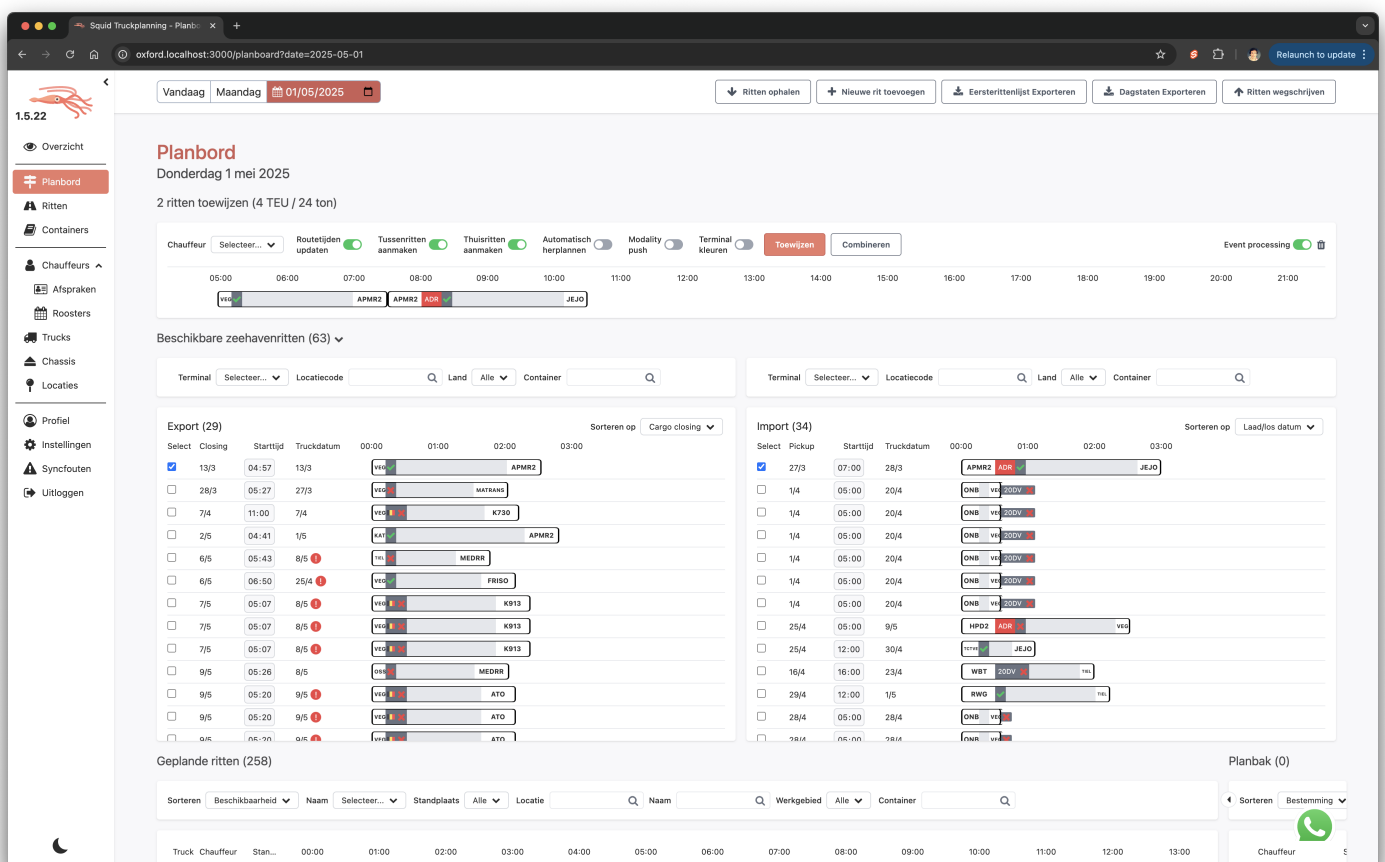


Figure 5.1: GUI in Action for Manual Port Planning

5.2 Postgres Database

As we have seen in our entity relationship diagram on Figure 4.2 our data is very relational. For that reason alone, it would already make sense to adopt a relational database, because execution of joins is easy and the referential integrity between records is ensured at a database level.

A non-relational database would be easier to scale. Scale, however, is not an issue. We can keep databases small by archiving all non-operational data and by setting up a separate database for each customer. This gives us a setup with a multi-tenant GUI connecting to several single-tenant databases. The database connection only happens server side and is strictly separated per tenant to avoid data leakage.

We choose PostgreSQL [34] because it is an open-source relational database that does offer us the option for non-relational data types (JSONB) and it has a built-in notification system that we can leverage for sending updates to the GUI over websockets.

5.3 Synchronisation Service

The synchronisation service is designed to handle all data processing tasks efficiently. It leverages BullMQ, a powerful Node.js library for managing queues and background jobs, to orchestrate and execute various tasks asynchronously. BullMQ enables the application to process large volumes of data in a scalable and reliable manner, ensuring that tasks are executed in the correct order and with retry mechanisms in case of failures.

Tasks can be triggered on demand through the exposed API but recurring tasks are also scheduled using cron jobs. These cron jobs are responsible for triggering specific workflows at predefined intervals, such as syncing data, cleaning up outdated records, or generating reports. This scheduling capability ensures that critical operations are performed automatically and consistently without manual intervention. The full data flow is visualised in the Sequence Diagram in Figure 4.4.

A key feature of the system is its ability to process data for multiple tenants using adapters. Each tenant may have unique requirements or data formats, and the application uses a modular adapter pattern to handle these differences .

The service was written in TypeScript which enhances the application's maintainability and reliability. TypeScript's static typing ensures that potential bugs are caught during development.

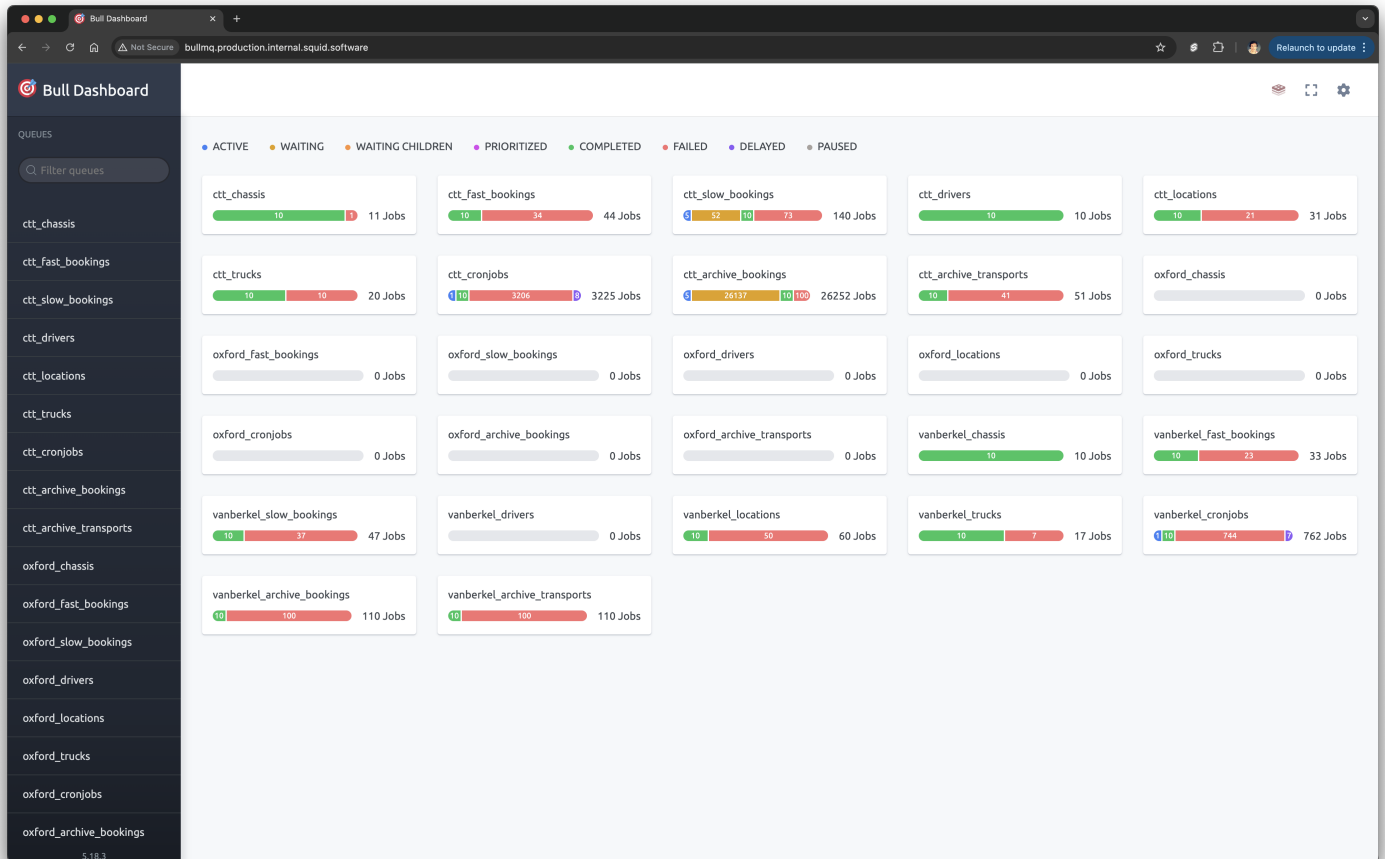


Figure 5.2: BullMQ Dashboard Displaying the Various Queues and Statuses of Jobs

5.4 OSRM Routing Service

We soon learned that much information does not exist digitally but only resides in the heads of planners. Driving times are a great example of this. A transport planner will tell you without thinking that it takes approximately 2 hours to drive from Veghel to the Rotterdam World Gateway terminal at the Maasvlakte. Google Maps will underestimate this by 30 minutes because it calculates with car speeds. To solve this problem, we build a service on the Open Source Routing Machine (OSRM) [35]. OSRM is a codebase for open-source routing that allows you to use publicly available maps and write a custom vehicle profile for route calculation. We have set this up as a microservice so we can calculate distances and driving durations by truck on-demand.

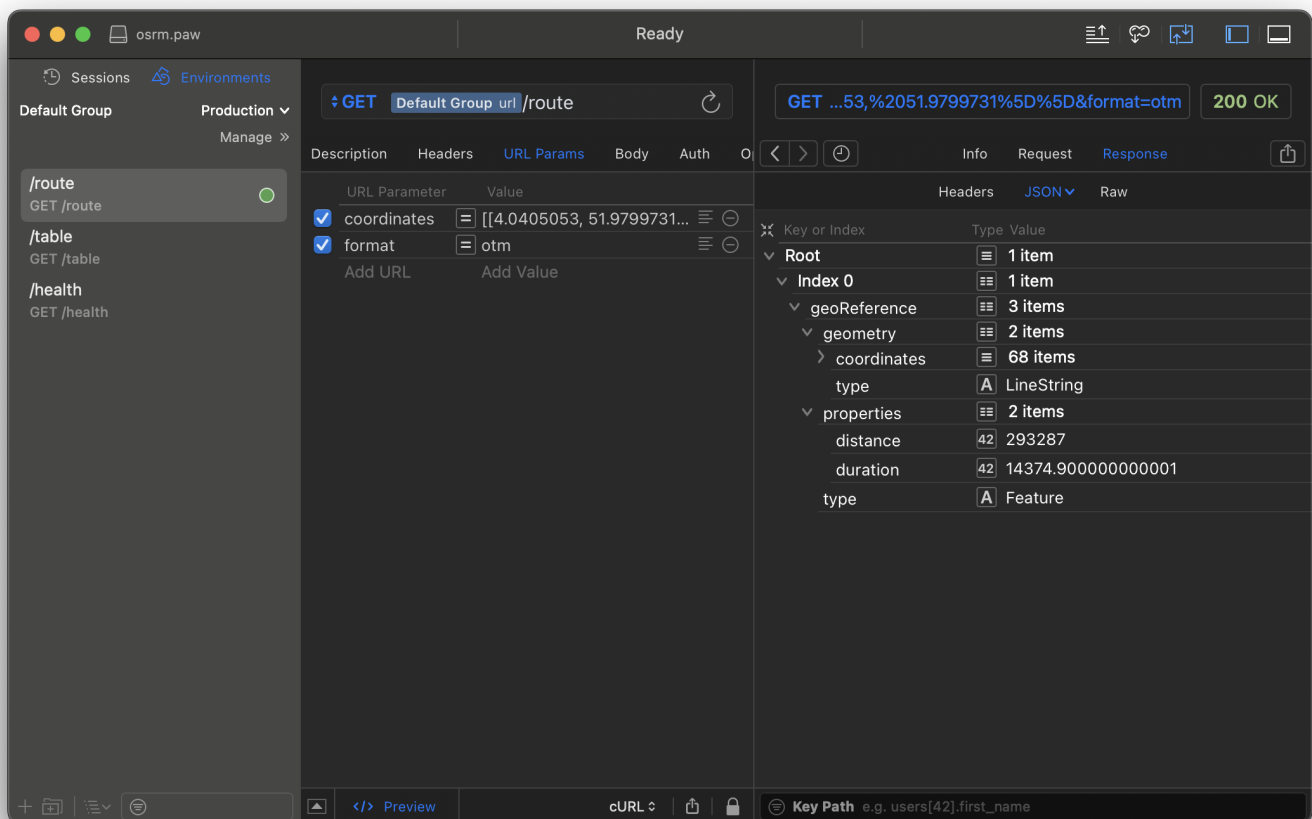


Figure 5.3: OSRM API Returning a Requested Route in OTM Format [31]

5.5 Geocoder Service

The ERP-system of Van Berkel Logistics stores location addresses but no GPS-coordinates, which is what the OSRM Routing Service requires as input. To solve this we build a separate microservice in Vanilla JS that checks the geocoding API's of Google [17] and HERE [20] to enrich our location data with GPS-coordinates.

5.6 Algorithm Development

With the main components in place (and deployed for operational use) we can start focusing on algorithm development. In this section we cover the data transformation pipeline we build that transforms the data into algorithmic input, and we discuss and reflect upon the implementation of both algorithms. Before starting, we should point out that we made the decision to build both algorithms from scratch. There is a temptation to cast the problem in such a way that off-the-shelf solvers can be used to find a quick solution, but this almost inevitably leads to leaving out constraints that matter in the real world. In addition, building from scratch ensures true understanding of what happens in the internals of the algorithms which makes it easier to expand and adapt them in the future.

5.6.1 Data Transformation

A Python data transformation pipeline was written to transform the initial anonymised data (as described in Section 3.4). This includes filtering out unwanted data and parsing the remaining records into a format that is more efficient for evaluation and computing matches between transports and drivers.

Time Normalisation

All temporal data is converted into "minutes since midnight" for computational efficiency. For example, 05:00 AM is represented as 300 minutes. This simplifies calculations of durations, overlaps, and time windows without complex date and time manipulations.

Driver Data Processing

The script applies several filtering steps to the driver data. Unavailable drivers, drivers assigned to other tasks (such as manning a crane), and drivers exclusively working in port areas are excluded. The resulting driver structure is as follows.

```
{
  "id": "driver_123",
  "name": "John",
  "startLocation": "VEG",
  "endLocation": "VEG",
  "startTime": 480,          // 08:00 in minutes
  "endTime": 1020,          // 17:00 in minutes
  "sodOvertime": -60,       // Start overtime allowance
  "eodOvertime": 60,        // End overtime allowance
  "hazardousGoods": true,
  "assignedTransports": []
}
```

Transport Data Processing

Transport data undergoes slightly more complex transformations. First we apply some filtering: We filter out any child transports from merged transport combinations, remove truck transfers between terminals without load, filter out transports to port areas, and we exclude transports with a midnight delivery. The last filter step matches operational practice, these transports are awaiting confirmation from the customer and are likely to be moved to the next day.

Secondly, the pipeline calculates delivery windows considering, by considering the planned delivery time, the total driving duration, the handling duration at each stop, and location-specific allowances for earliness and lateness. For decouple transports, the driving duration is doubled to account for the return trip to the home terminal. The final transformed transport structure shown below.

```
{
  "id": "transport_456",
  "startLocation": "VEG",
  "endLocation": "OSS",
  "minWindow": 540,          // Earliest start
  "delivery": 600,           // Target delivery
}
```

```

"maxWindow": 660,          // Latest end
"duration": 120,           // Total duration
"hazardousGoods": false
}

```

Business Constraints

This transformation process encodes several business rules into the data:

1. **Home terminals:** Only include transports starting or ending at the inland terminals of Veghel, Cuijk, Oss, and Tiel.
2. **Transport types:** Handle different transport types (delivery, pickup, transfer) with specific rules.
3. **Location constraints:** Respect location-specific earliness and lateness allowances.
4. **Duration calculation:** Account for both driving and handling time at each stop.

Output Generation

For each date, the transformation process generates two JSON files:

- `drivers.json`: Contains available drivers with their constraints.
- `transports.json`: Contains valid transports with calculated time windows.

These standardised files serve as input for both the multi-agent and greedy search algorithm. This transformation process is crucial for reducing the problem space to manageable dimensions while maintaining all essential business constraints. Both algorithms also output their solution in the same format, ensuring consistent evaluation and comparison of results.

5.6.2 Single-Agent Greedy Search

The single-agent implementation uses a greedy search strategy to assign transports to drivers. The algorithm processes transports sequentially, making locally optimal decisions for each assignment without backtracking. The advantage of this is performance as the search space gets smaller after each decision.

Implementation Details

Before doing any computation the drivers and transports are sorted:

- Drivers are sorted by their earliest availability and capabilities. Hazmat drivers are sorted on top to ensure they get a schedule that includes hazmat transports. Sorting drivers by start time gives by priority to drivers that start early.
- This is a good idea because we also sort transports by the start of their delivery window. Early transports have a high chance of getting assigned to drivers with an early start.

With sorted drivers and transports the algorithm loops over each driver and conducts a beam search with width k to find the k best schedules for this truck driver. A schedule is a sequence of non-overlapping transports that respect time windows and driver constraints. We then greedily assign the best schedules to the current driver and continue to the next driver. In pseudocode:

```

availableTransports := all transports

for driver in drivers:
    feasibleTransports := filter(
        availableTransports,
        isFeasibleFor(driver)
    )

    if feasibleTransports is empty:
        continue

    beams := [empty schedule]
    for t in feasibleTransports:
        newBeams := []

        for b in beams:
            if isFeasibleAddition(t):
                newBeam.transports := beam.transports.append(t)
                newBeam.metrics := scoreBeam(newBeam)
                newBeams.append(newBeam)

        beams := topKSort(beams + newBeams, beamWidth)

    bestSchedule := beams[0]
    driver.assignedTransports := bestSchedule

    for t in bestSchedule.transports:
        availableTransports.remove(t)

```

The actual algorithm was written in TypeScript. Automatic type checks catch a lot of bugs even prior to testing the code and Node.JS makes it very easy to pause the code whenever an asynchronous call needs to be made.

Greedy Search Reflection

There are some advantages, limitations, and potential improvements for this search algorithm.

Advantages:

- **Computational efficiency:** Each driver and transport is processed exactly once, while the beam width k effectively controls both search space and memory usage. The search space is reduced at each step, preventing exponential growth.
- **Solution quality:** The algorithm explores multiple promising paths through its beam width approach, while maintaining the best partial solutions for future consideration. The sorting mechanism prioritizes both early transports and hazmat transports.
- **Implementation simplicity:** The algorithm features a straightforward logic flow that is easy to maintain and debug. Additionally, it remains simple to modify or add new constraints as needed.

- **Deterministic behaviour:** The algorithm produces consistent results for the same input and exhibits predictable performance characteristics.

Limitations: Limitations are mostly related to the quality of the solution. By making greedy decisions we are likely to get stuck in local optima and less likely to approach the global optimum.

- Early decisions may lead to poor utilisation of later drivers
- There is risk of leaving transport unassigned or missing better transport combinations due to early suboptimal choices.
- Limited ability to backtrack when better options are discovered (we only keep old beams when their score ranks them in the top k of beams).
- Beam width significantly impacts solution quality. A too narrow width will miss good solutions. A too wide width leads to computational overhead with diminishing returns.

Potential improvements:

- **Preprocessing:** Instead of sequential sorting by earliest start we can group transports by how well they match up. When considering a driver we can sort by the earliest start of the group.
- **Look-ahead mechanism:** We can implement some form of forward checking, for example by adding transports to a beam based on the score yielded from adding the best fitting follow-up transport.
- **Multiple passes:** We can try different orderings of drivers and transport and keep the best overall solution

5.6.3 Multi-Agent System

The multi-agent implementation has two core components:

1. A coordinator that orchestrates the assignment process by managing driver agents, providing driver agents with information, collecting driver bids, and assigning transports to the highest bidder.
2. Each driver is represented by an agent that evaluates potential schedules and submits bids for individual transports to the coordinator.

Implementation Details

The multi-agent implementation uses the exact same scoring mechanism for evaluating schedules as we applied to the greedy search algorithm (see Section 2.2). There are some notable differences, however. Unlike the greedy approach, drivers are unsorted while transports are sorted on earliest delivery time instead of earliest possible start. Transports are evaluated and assigned on a case-by-case basis instead of as complete schedules. Rather than computing k best schedules, the system conducts a search for the next best alternative transport. Finally, assignments are made by evaluating the bids of all drivers.

The coordinator plays a central role in orchestrating the entire process. It starts by initialising all driver agents and sorting the available transports. It then manages the communication flow by sending transports one-by-one to the drivers, collecting their individual bids, and ultimately assigning each transport to the highest bidder.

Each driver agent maintains its own schedule consisting of won bids on transports. When the driver agent receives a request from the coordinator to evaluate a transport, it follows a specific evaluation sequence. First, it checks the feasibility of adding this transport to the current schedule. Then, it calculates the score of the new schedule and determines the score of the next best alternative. Based on these calculations, it submits a bid by computing the difference between the two scores, which represents the marginal utility of accepting this transport over the next best alternative. Finally, it monitors the result and either updates its schedule or discards the transport accordingly.

In sequential pseudocode:

```

availableTransports := all transports
driverAgents := initialize_driver_agents(drivers)
sort availableTransports by delivery time

for t in availableTransports:
    bids := []

    for driverAgent in driverAgents:
        currentSchedule := driverAgent.get_schedule()

        if not isFeasibleFor(driverAgent, t):
            continue

        withTransport := create_schedule_branch(
            [t | currentSchedule],
            driverAgent
        )
        scoreWith := score_branch(withTransport, driverAgent)

        bestAlternative := find_best_alternative(
            driverAgent,
            currentSchedule,
            availableTransports - {t}
        )

        bidValue := scoreWith - bestAlternative
        bids.append((driverAgent, bidValue))

    if bids is not empty:
        winningAgent := argmax(bids, bidValue)
        winningAgent.accept_transport(t)

    for driverAgent in driverAgents:
        if driverAgent != winningAgent:

```

```

        driverAgent.reject_transport(t)
    else:
        mark t as unassigned

return final assignments

```

The actual algorithm was written in Elixir. This proved quite a challenge because the author had zero experience with this language prior to this project. We still decided to pursue Elixir because earlier experiences of attempting to write multi-threaded concurrent systems in Node.JS and Python leave a painful memory. Elixir is build on the BEAM (the Erlang VM) which was specifically designed for concurrent distributed systems with lightweight processes that communicate via message passing. Simply put, it is the tool for the job.

Multi-Agent Reflection

Advantages:

- **Distributed decision making:** Each driver makes local decisions, with no central optimisation required. This approach naturally enables parallelisation of the decision-making process.
- **Flexibility:** The system makes it easy to add new drivers, while effectively handling dynamic changes and adapting to local constraints.
- **Transparency:** The clear bidding process makes decisions easy to understand, providing effective human oversight of the system.

Limitations:

- **Sequential processing:** Transports processed one at a time. As a result, the order significantly affects the results. This may lead to a suboptimal solution because we may miss out on better combinations.
- **No look ahead:** Scores are calculated by adding single transports to the current schedule. By calculating a full branch we may get better results.
- **Computational overhead:** Each transport is evaluated by all drivers and followed by a bidding process, which is a lot more expensive than our greedy approach.

Potential improvements:

- **Concurrent processing:** The current implementation requests bids from drivers sequentially, where running the bidding in parallel would provide an easy performance gain. Rather than processing transports sequentially, we can run them concurrently, though this complicates the process as we need to implement batch bidding and find a way to resolve conflicts between schedules that contain the same transport. We will likely need multiple rounds of negotiation.
- **Enhanced bidding:** We can reduce computation by keeping a cache of how well alternatives fit with the current schedule. Future opportunities could be considered by adding a look-ahead mechanism or adapting the branch generation from the greedy algorithm. Additionally, bid values could be made more strategic than simple marginal utility by including consideration of how well a transport fits with future high-utility opportunities.

- **Backtracking:** In the current implementation transports stay with the driver once assigned. There is no mechanism to recover from poor decisions.

6 Evaluation

This chapter is all about planners versus the machines, we are not evaluating cooperation, but we are trying to measure how well our two algorithms perform compared to human planners. To ensure scientific rigor in this evaluation, we have:

- Developed standardised data transformation pipelines, see Section 5.6.1.
- Defined an objective function in Equation 4.1.
- Developed a standardised scoring mechanism for driver schedules explained in Section 6.4 which based on this objective function.
- Established clear metrics for measuring performance that explained below in sections 6.2 and 6.3.
- Collected data from human planners for five subsequent weekdays.

6.1 Problem Setup and Methodology

The planners were asked to not use autoplan for the duration of one week between Monday 28th of April and Friday 2nd of May. The original data of the transport plans they generated were exported and anonymised (see Section 3.4). The anonymised data were transformed into the same format as the output of both algorithms (see Section 5.6.1).

This allows us to compare the three transport-driver matching solutions using a comprehensive set of Key Performance Indicators (KPIs). The selected KPIs measure both operational efficiency and constraint satisfaction. These metrics are calculated for both individual drivers and the overall solutions generated by the three methods.

6.2 Individual Driver KPIs

Work time utilisation: Work time utilisation measures the proportion of a driver's available time that is spent on active tasks:

$$\text{utilisation} = \frac{\text{active_time}}{\text{end_time} - \text{start_time}}$$

where active_time includes both driving and handling duration of assigned transports.

Idle time: Idle time represents gaps between transports where a driver is not actively working:

$$\text{idle_time} = \sum_{\text{gaps}} (\text{next_transport_start} - \text{current_transport_end})$$

Overtime: Overtime is calculated in two components:

- Start of Day (SOD) Overtime: Work before the driver's scheduled start time
- End of Day (EOD) Overtime: Work after the driver's scheduled end time

Delivery time deviation: Average deviation from planned delivery times:

$$\text{avg_deviation} = \frac{\sum |\text{actual_delivery} - \text{planned_delivery}|}{\text{transport_count}}$$

6.3 Overall Solution KPIs

Assignment rate: Measures the proportion of transports successfully assigned:

$$\text{assignment_rate} = \frac{\text{total_transports} - \text{unassigned_transports}}{\text{total_transports}}$$

Driver utilisation rate: Proportion of available drivers being used:

$$\text{driver_utilisation} = \frac{\text{total_drivers} - \text{unused_drivers}}{\text{total_drivers}}$$

Average metrics: Solution-wide averages calculated only for active drivers (see Section 6.2):

- Average work time utilisation
- Average idle time
- Average delivery time deviation
- Average overtime

6.4 Scoring system

The overall quality of the solution is evaluated using a weighted scoring system that is based on our previously defined objective function (see Equation 4.1):

- Overtime weight: 0.1
- Idle time weight: 0.2
- Active time weight: 0.3
- Delivery time deviation weight: 0.3
- Hazardous goods handling weight: 0.1

This outputs a score between 0 and 1 that is multiplied with 100 to get a feel for how good the solution is. A score can be computed both for the assigned schedules of individual drivers and the system-wide averages to get a feeling how good the solution really is.

The weights are chosen somewhat arbitrarily but reflect the trade-off that has to be made between the different objectives that make up a good planning (as described in Section 2.2):

- Driver preferences: Represented by the overtime factor
- Capacity utilisation: Represented by the factors idle time, active time, and hazardous goods handling
- On-time performance: Represented by the delivery time deviation factor

6.5 Problem size

We have data available for five days. The size of the problem varies slightly for each day and is visualised in the table below. Note that there were unfortunately no hazmat transports on any of the days.

Metric	Monday	Tuesday	Wednesday	Thursday	Friday
Drivers	53	56	52	49	56
Hazmat Drivers	2	2	2	1	3
Transports	244	257	234	222	245
Hazmat Transports	0	0	0	0	0

Table 6.1: Problem size per day

6.6 Results

Calculation Time

The calculation time needed varied a bit over the days, with the Human Planner showing an outlier on Friday, Greedy Search on Tuesday, and Multi-Agent on Thursday. Note that the calculation for human planners is measured in minutes, where both the calculation time of Greedy Search and Multi-Agent is measured in milliseconds.

Day	Human Planner (min)	Greedy Search (ms)	Multi-Agent (ms)
Monday	282	10	83
Tuesday	328	55	92
Wednesday	316	9	85
Thursday	445	9	62
Friday	191	11	72
Average	312	19	79

Table 6.2: Calculation time per method per day

There are a few caveats to make about measuring the calculation time for human planners. The time needed was measured by extracting from the logs when they started and finished assigning transports. On average this process took five hours, on Thursday they finished in three. However, they do not just sit down and complete the task. The measurement also includes answering emails, making phone calls, toilet breaks, getting a coffee, eating a sandwich or chatting to the author. We considered reducing the measurement with some arbitrary value. However, planning was not conducted by a single planner. One planner took care of the drivers departing from Veghel, Cuijk, and Oss, another planned Tiel, and one some days the dispatchers (a planner that is primarily responsible for ad hoc problems and replanning during the day) at Cuijk and Oss made some changes halfway to improve the planning that affected their terminal.

We can conclude that algorithm planning is in the worst scenario 15.000 times faster than human planning. This, however, only holds if a human planner no longer has to make adjustments to the algorithmic solution.

We can also conclude that Greedy Search is around 8 times faster than Multi-Agent (leaving Tuesday out of the averaging).

Solution Quality

Let's take a look at the quality of the three solutions. In Section 6.4 we developed a method to score a solution. Figure 6.1 shows the overall solution score for each method for each day. The average scores were:

- Human Planner: **57%**
- Greedy Search: **80%**
- Multi-Agent: **89%**

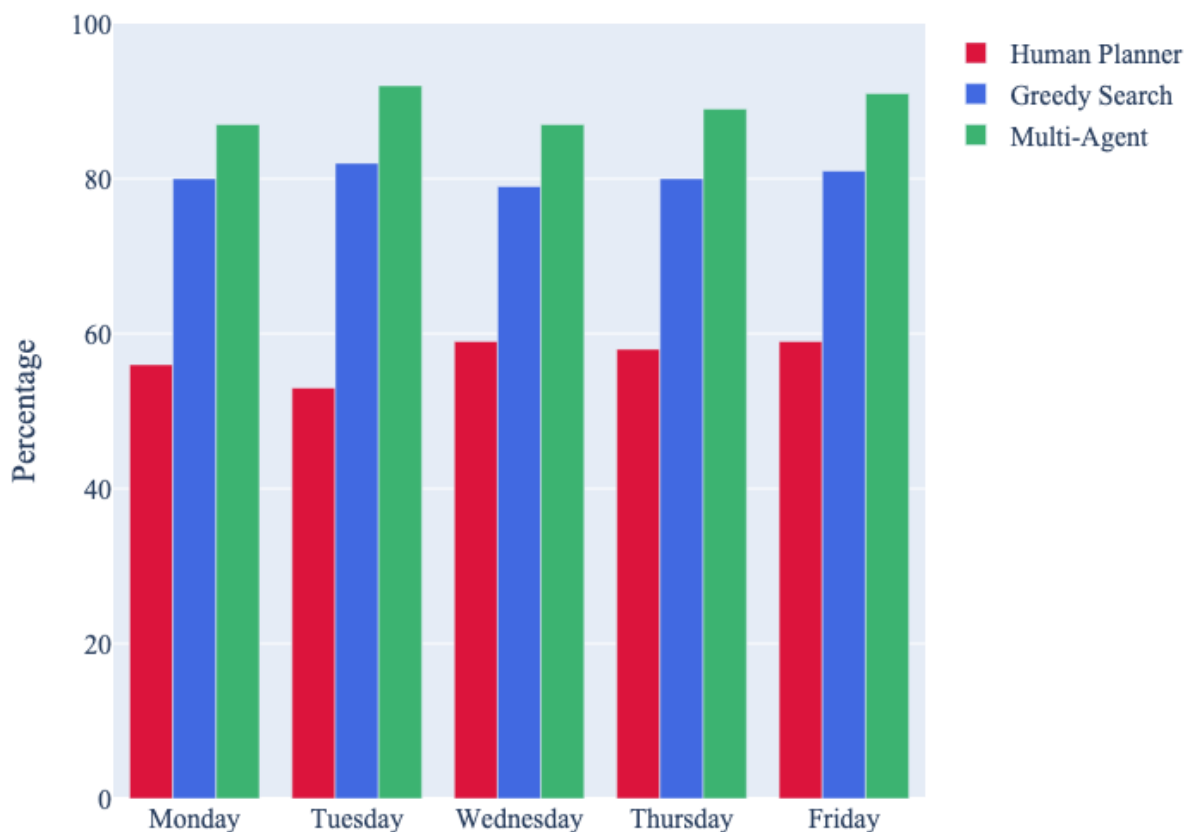


Figure 6.1: Solving a network graph

It is not that surprising that Multi-Agent outperforms Greedy Search as the algorithm is more advanced and takes more computational time to find a solution, but how can we explain the performance of human planners? We will look at several averaged solution level KPIs (see Section 6.3) to answer this question:

- Assignment Rate
- Driver Utilisation Rate
- Averaged Schedule KPIs:
 - Work Time Utilisation
 - Idle Time
 - Overtime
 - Delivery Deviation

Assignment Rate

Day	Human Planner	Greedy Search	Multi-Agent
Monday	59 / 76%	43 / 82%	31 / 87%
Tuesday	50 / 81%	44 / 83%	31 / 88%
Wednesday	42 / 82%	46 / 80%	51 / 78%
Thursday	35 / 84%	48 / 78%	35 / 84%
Friday	58 / 76%	44 / 82%	28 / 89%
Average	49 / 80%	45 / 81%	35 / 85%

Table 6.3: Daily unassigned transport count / Assignment rate

It is a bit surprising that planners leave 20% of transport unassigned. This, however, has practical reasons. Figure 2.3 shows that the peak of transport demand is in the morning. There is little incentive to assign transports late in the afternoon, because there will be surplus capacity during that time period. Secondly, last-minute bookings are not accounted for in the capacity plan. These transports, however, will only be executed if there happens to be some availability during the day. In contrast to the human planner, Greedy Search and Multi-Agent will only leave transports unassigned if there is no driver with availability to execute them.

Driver Utilisation Rate

Driver utilisation rate turns out to be a very uninteresting statistic. As Table 6.4 shows we typically use all drivers. This is by design, planners use charters to scale their fleet size to the required capacity. Last-minute downscaling does not happen very often, drivers are put on calamity duty to pick up the problems. Excess capacity is usually not a very big problem as excess drivers can be used to pickup empty containers at customers, execute ad hoc transports, take over transports from delayed drivers, rebalance container stock between terminals, execute transfers (sometimes containers are repositioned for a later transport by barge) or even just get trucks washed and cleaned.

Day	Human Planner	Greedy Search	Multi-Agent
Monday	3 / 94%	1 / 98%	1 / 98%
Tuesday	1 / 98%	0 / 100%	0 / 100%
Wednesday	3 / 94%	0 / 100%	1 / 98%
Thursday	1 / 98%	0 / 100%	0 / 100%
Friday	9 / 84%	0 / 100%	0 / 100%
Average	3 / 94%	0 / 100%	0 / 99%

Table 6.4: Daily unassigned driver count / Driver utilisation rate

Averaged Schedule KPIs

The KPIs in Figure 6.2 and Table 6.5, however, provide some interesting statistics. We will discuss and elaborate on each KPI below.

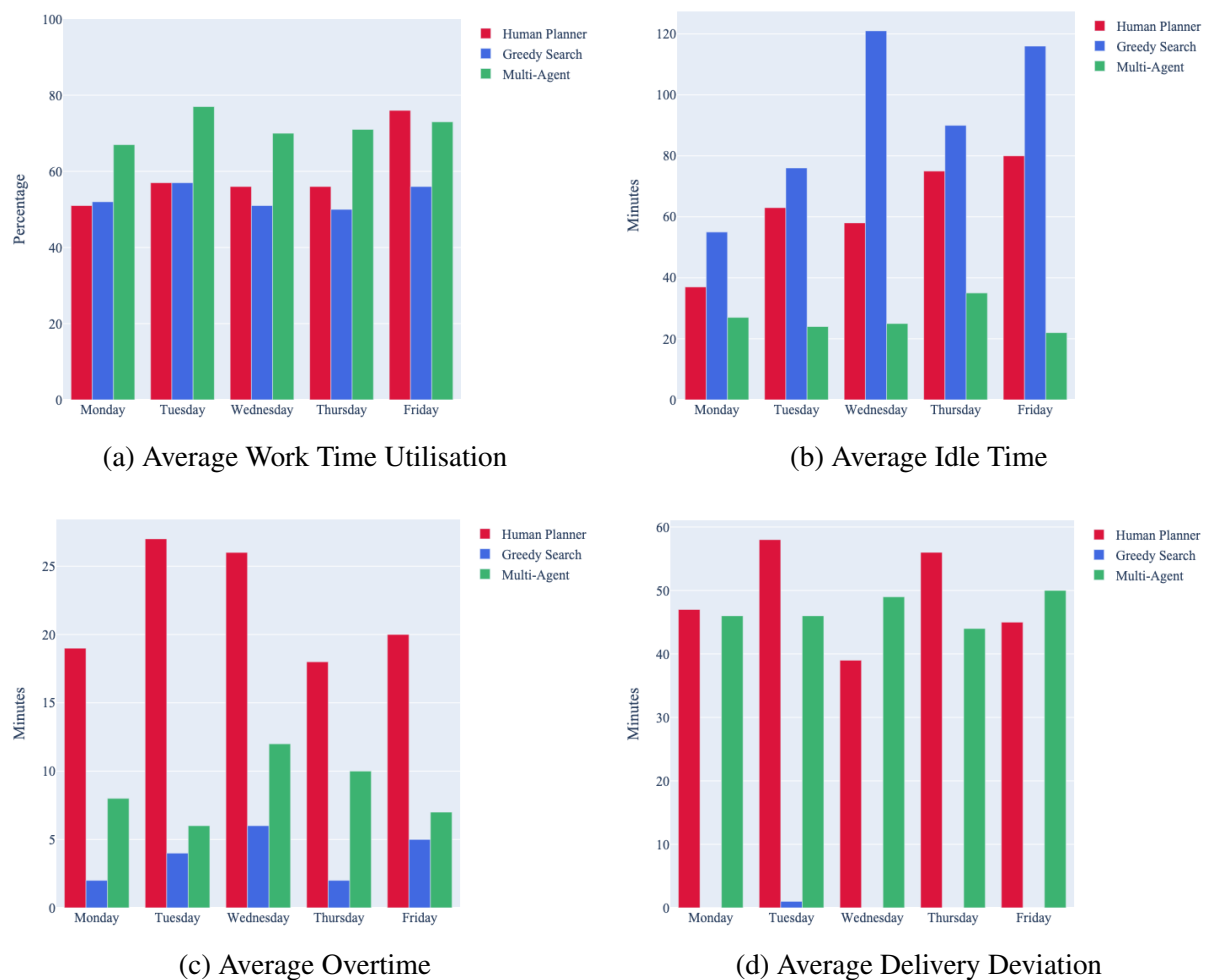


Figure 6.2: Key Performance Indicators

- **Average Work Time Utilisation:** Multi-Agent outperforms the other methods for this KPI. This is particularly good given that there is little overtime and little idle time compared to the other methods. This is a direct result of the higher assignment rate.

KPI	Human Planner	Greedy Search	Multi-Agent
Average Work Time Utilisation	59%	53%	72%
Average Idle Time	63	92	27
Average Overtime	22	3.8	8.6
Average Delivery Deviation	49	0	47

Table 6.5: KPI comparison between different methods

- **Average Idle Time:** Greedy Search performs badly, but even human planners have twice as much idle time compared to Multi-Agent. My expectation was the opposite, but what is happening with Greedy Search is that the first drivers get very efficient tight schedules assigned which results in pretty bad schedules for the later drivers. Very limited idle time is problematic because one delay will cause a schedule to be infeasible. Idling for 30 minutes a day sounds reasonable though as the average driver has about 4 transports assigned.
- **Average Overtime:** The overtime of Human Planners looks excessive with an average of 22 minutes per driver and seems unnecessary given the results of the other methods.
- **Average Delivery Deviation:** This is a very interesting. Greedy Search sticks closely to the delivery time despite being branches being scored similarly to Multi-Agent. Multi-Agent seems to make maximum use of the delivery windows despite being penalised for this in the same way as Greedy Search is. This is an indication that we need to alter either the bidding process or the scoring of branches. The deviation of the human planner has an explanation which will discuss below.

Discussions with the planners revealed that there exists such a thing as spread cargo agreements. An example of this is shown in Figure 6.3. This container has a delivery time of 9:30, yet is planned to arrive at 15:40, more than 6 hours late. The reality is that for these customers times are indication of preferred container ordering. The actual agreement is that they want n containers every h hours. This particular customer wants 2 containers every 90 minutes, which roughly corresponds with the schedule shown in Figure 6.3. We should aim to capture these agreements in our data model and either apply the spreading in the preprocessing prior to running our algorithms or update the algorithms. A further complication is that these addresses are usually serviced with dedicated trucks. A handling delay in one of the transports will cascade on to the next transports. The customer knows this and this not considered to be a problem.

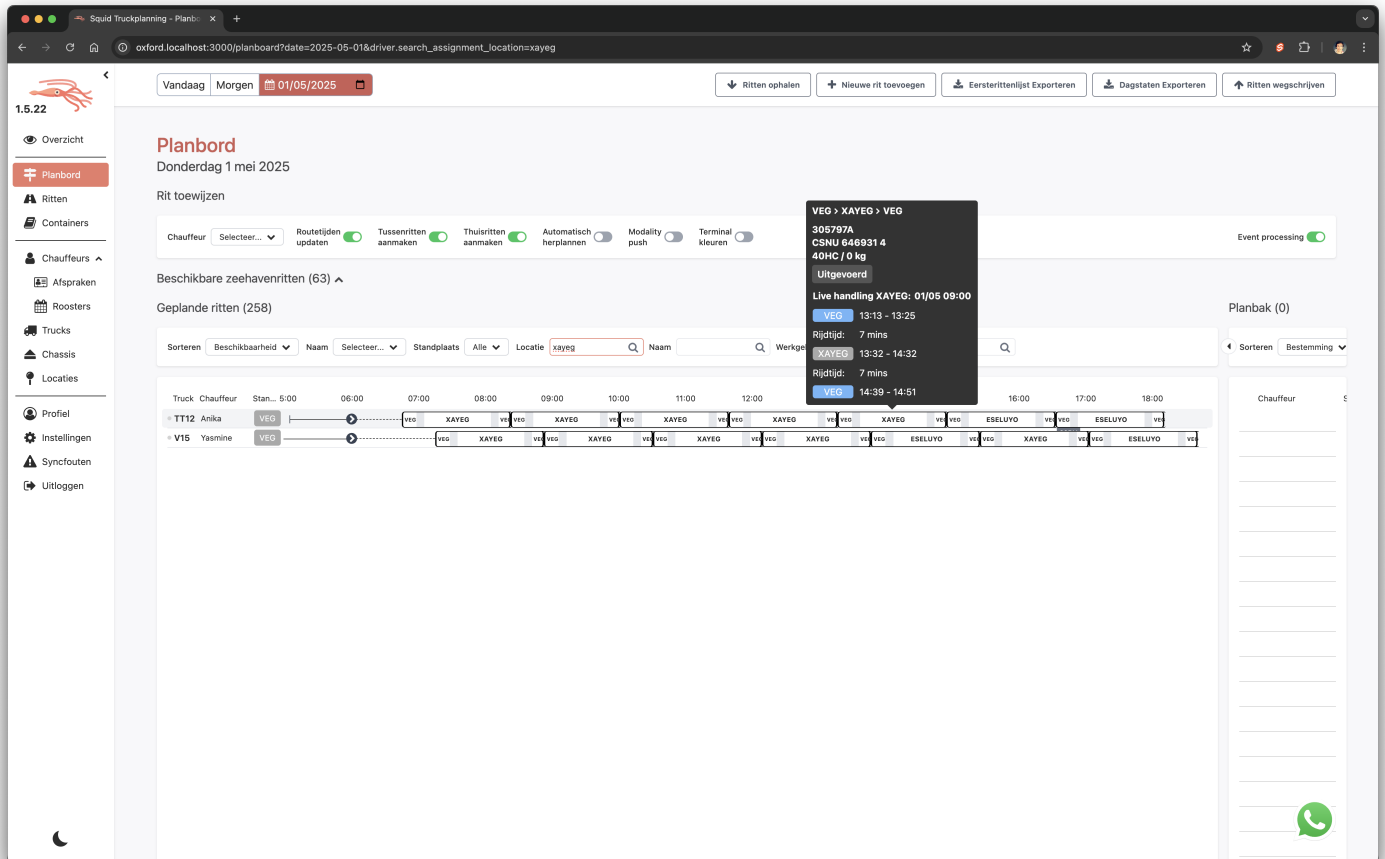


Figure 6.3: Customers with Spread Cargo Agreements

The evaluation results demonstrate both the potential and limitations of algorithmic planning approaches in real-world transport logistics. Three key findings emerge:

1. **Computational Efficiency:** Both algorithmic solutions dramatically outperform human planning in terms of speed, with the greedy search being approximately 15,000 times faster than manual planning and 8 times faster than the multi-agent approach. This satisfies our non-functional requirement NFR4 for generating planning proposals within 2 minutes.
2. **Solution Quality:** The algorithmic approaches consistently produced higher-quality solutions:
 - Multi-agent system: 89
 - Greedy search: 80
 - Human planners: 57

However, there are some caveats to these scores. Human planners deliberately leave capacity unassigned to handle real-world dynamics like last-minute bookings and disruptions and spread cargo agreements result in lower score while not hurting customer satisfaction.

3. **Practical Implications:** While algorithms excel at optimising measurable metrics, human planners demonstrate valuable strategic thinking not captured in our scoring system.

For instance, the human practice of maintaining slack capacity (shown by their 80% assignment rate) reflects an understanding of operational realities that our algorithms do not yet consider.

These findings suggest that the best approach may not lie in replacing human planners but in enhancing their capabilities. The algorithms can rapidly generate high-quality initial solutions, while human planners can apply their expertise to already adjust plans for known future events, maintain strategic capacity buffers, handle exceptions and special cases, and make real-time adjustments as conditions change.

This aligns perfectly with our ultimate goal of "enhancing the capabilities of human transport planners by reducing cognitive load, increasing operational efficiency, and improving decision-making quality." See Section 3.2. The evaluation demonstrates that we have created a system that can generate technically sound planning proposals (shown by the high solution scores) while leaving room for human expertise in handling the dynamic nature of transport planning.

Future work should focus on incorporating the strategic aspects of human planning into the algorithmic approaches, particularly in areas such as dynamic capacity management, predictive buffer allocation, multi-modal decision making, and learning from human planner adjustments.

In conclusion, while the algorithmic solutions show impressive performance in optimising standard metrics, their true value lies in complementing rather than replacing human expertise. This evaluation validates our approach of developing tools that enhance, rather than automate, the transport planning process.

7 Conclusion

7.1 Revisiting Our Goals

This project set out with an ultimate goal to "design a system that enhances the capabilities of human transport planners by reducing cognitive load, increasing operational efficiency, and improving decision-making quality." To achieve this, we established three explicit goals (see Section 3.2):

- G1** Capture the problem in a fitting data model
- G2** Design and implement an appropriate software architecture
- G3** Explore and evaluate algorithmic solutions for automating planning proposals

Looking back, we have successfully delivered on these goals. Our data model captures most of the complexities of container road transportation, as evidenced by its successful implementation at Van Berkel Logistics. The software architecture, despite some challenges, has proven robust enough to handle real-world operations. Our algorithmic solutions demonstrate significant improvements over manual planning in both speed and solution quality.

7.2 Meeting Requirements

Our functional requirements (see Subsection 3.3.1) were largely met:

- ✓ **FR1:** Planners can effectively manage constraints through the GUI. They do so every day, GUI screenshots are included in Appendix Section A.8.
- ✓ **FR2:** The system provides clear insights into capacity versus demand. There is good insight in whether tomorrow's transport capacity is sufficient to meet transport demand, see Figure 2.3. This, however, assumes that we know the delivery times of transports which is only the case a few days in advance. A better forecasting model that extends to weeks in advance was marked out of scope in Section 3.1.
- ✓ **FR3:** Manual assignment capabilities are preserved. In fact the planners used the Squid prototype GUI developed as part of this dissertation to generate the benchmark data we used to evaluate our algorithms.
- ✓ **FR4:** The system successfully signals potential issues. This is a topic we have neglected to discuss because there is little technical challenge. The screenshot of the planboard in Appendix Section A.8 shows some orange transports, these are transports with alerts attached that the planner should take a look at. In addition, when using manual planner the planner is alerted when he attempts to do things he should not do, like assigning a non-hazmat driver to a hazmat transport.

- ✓ FR5: Algorithm explainability is ensured by defining a clear objective function in Equation 4.1 and by ensuring that all algorithm generated schedules can be scored using a scoring function derived from this objective function (see Section 6.4). The algorithms have not yet been wired up with the GUI, but it is the plan to visualise these scores and recalculate them instantly for both the affected driver and the overall solution whenever the planner wants to manually override an assignment.

For non-functional requirements (see Subsection 3.3.2) we successfully met half our requirements and partially met the other half:

- ✓ NFR1: Multiple planners can work simultaneously. Our choice for a PostgreSQL database in combination with a custom written Socket.io integration in SvelteKit allows to instantly transmit the actions from one planner to the screen of a second planner (see Section 5.1).
- ~ NFR2: Initial load times meet the 1-minute target, but some performance issues remain. When planner work for extended periods with the GUI or when large numbers of updates are synced from the ERP-system to the database interaction lag starts to occur. The GUI likely has a memory leak that we should track down and we need to develop a better method to only send the relevant updates back to the frontend over websockets.
- ~ NFR3: Our goal was near instead ERP-synchronisation. The synchronisation works but is based on a pull-based mechanism that queries the ERP-system for updates on a 1-minute interval. This is acceptable to human planners, but it is not "near-instant". This could be improved with push-based integration, but implementation depends on willingness of the IT supplier of the ERP-system.
- ✓ NFR4: Algorithmic proposals are generated well within the 2-minute limit. We expected needing large amounts of time to find a near optimal solution. The developed methods do so in the order of milliseconds which means we far exceeded our expectations and have plenty of computational time left for algorithm improvement.

7.3 Project Reflections

7.3.1 What Went Well

The project has achieved several significant successes. Most notably, the system is in daily use at Van Berkel Logistics, providing concrete proof of its practical value. Strong collaboration with planners led to high user acceptance, frequent feedback, and an increase in data quality. We also managed to successfully capture real-world constraints in our data model and transform these into algorithm inputs. Both algorithms have not been deployed to production, but they have been tested on real data and demonstrated impressive performance improvements, operating up to 15,000 times faster than manual planning. Finally, and perhaps most significantly, the project has evolved beyond academic research to become a viable business with Squid.

7.3.2 What Could Have Been Better

Several challenges emerged during the project's development. To start with, the initial scope proved perhaps too broad. We could have written a full dissertation about the GUI, about integration with legacy ERP-systems, about microservices, or about the developed algorithms.

Yet we attempted to combine all this work in a single dissertation to present a full software solution.

Building all these components and implementing them in the real world led to several challenges. The launch of a second customer created unexpected difficulties with our multi-tenant frontend architecture. Performance has also been a major challenge. Our choice of PostgreSQL as the database may have impacted this, as PostgreSQL is not optimized for high throughput of events. On the GUI-side we improved performance with a variety of technical solutions including DOM virtualisation, Protobuf data transfers, and minimising recalculation of global variables containing data stores..

The synchronisation service required a complete rebuild, evolving from a simple cron-based processing engine to a full-featured queuing mechanism to ensure rapid processing of immediate updates while handling future bookings at a lower priority. Finally, building algorithms from scratch and choosing Elixir for the Multi-Agent System was intellectually rewarding and arguably the right approach, but it also added unnecessary complexity and consumed considerable time. Better results might have been achieved with off-the-shelf solvers, though we do maintain that having a well-understood foundation for future expansion was worth the investment.

7.3.3 What I Would Do Differently

Looking back, several strategic changes would have improved the project's execution. Rather than tackling a complete system, focusing on a single component as a dissertation topic would have allowed for deeper analysis. The database architecture could have benefited from considering non-relational options for better performance, and the data model should have been designed with multi-modal transport in mind from the start. Implementation of push-based integration with external systems should have been prioritised earlier, along with greater emphasis on algorithm explainability and user feedback collection in the GUI. Additionally, minimising frontend data load should have been a primary consideration from the beginning rather than an optimisation challenge we had to solve later.

7.4 Key Takeaways

For practitioners and researchers in transport planning automation, four key insights emerge from this work. First, human-centric design is crucial. The most successful aspects of the system are those that enhance rather than replace human decision making. Second, algorithmic explainability matters. Both our multi-agent system and the simpler greedy approach generated good solutions, but these also immediately raised questions about on-time performance for Multi-Agent and about long idling times for Greedy Search. Having these KPIs built in helps in improving the algorithms and explaining their outputs to planners. Third, data quality improves with use. The system's implementation in the actual operation created a positive feedback loop that improved data quality. Finally, practical solutions beat perfect ones. Sometimes, a fast, good-enough solution that planners can adjust is more valuable than a slower, theoretically optimal one.

7.5 Key Takeaways

For practitioners and researchers in transport planning automation, four key insights emerge from this work. First, human-centric design is crucial—the most successful aspects of the system are those that enhance rather than replace human decision making. Second, algorithmic

explainability matters significantly; both our multi-agent system and the simpler greedy approach generated good solutions, but these also immediately raised questions about the on-time performance for Multi-Agent and long idling times for Greedy Search. Having these KPIs built in helps in improving the algorithms. Third, data quality improves with use, as the system’s implementation actually improved the quality of planning data, creating a positive feedback loop. Finally, practical solutions beat perfect ones—sometimes, a fast, good-enough solution that planners can adjust is more valuable than a slower, theoretically optimal one.

7.6 Future Work

Several promising directions emerge from this work, covering three main areas: algorithm enhancement, scope extension, and technical improvements.

In terms of algorithm enhancement, we need to improve the handling of dynamic changes, as our current approach can only recompute the entire schedule with different truck start locations. This can be done in a more sophisticated manner, especially for the Multi-Agent system. We must incorporate missing constraints to account for our second customer’s diverse chassis requirements, cross-border operations, and mandatory break requirements for longer regional drives. The system should also allow transfers between locations when work is scarce in certain terminals, we should remove the assumption of symmetric inbound/outbound driving duration, and add a bonus for padding time to absorb delays. Making the Multi-Agent truly concurrent and developing learning capabilities from planner adjustments are also crucial next steps for improving performance and solution quality.

Regarding scope extension, we should improve data preprocessing by automatically identifying candidates for “throughloading” and designing better solutions for planning transports with spread cargo agreements. Empty container tracking is a key area for improving operational efficiency. Our current assumption, which reflects operational practice, is that a driver will pickup an empty container after decoupling a full container. Better tracking would allow planning with pickups. Furthermore, the system should take pre- and post-transports into account while automatically tracking container availability. Finally, integration with truck onboard units for real-time GPS locations, port transport optimisation, demand forecasting integration, and expansion to multi-modal planning are all valuable future directions.

On the technical front, several improvements are necessary. We need to implement push-based ERP integration and optimise frontend performance. Data enrichment could be automated further, for example by retrieving axle loads, weights, and Euronorm classifications from the RDW (the Dutch Vehicle Authority). Finally, the developed algorithms need to be deployed to production as microservices in their own right, and their solutions and correspondings scores should be connected to the GUI.

I have not decided whether to move forward with Greedy Search or Multi-Agent. Both algorithms show potential and leave much room for improvement. The starting point would be to improve the pre-processing, evaluate the generated schedules in detail, and then tweak the scoring mechanism, and the internals of the algorithm. In addition, it would be very interesting to explore the potential of LLMs both for parsing human planner input and for explaining algorithm output.

7.7 Final Thoughts

This project demonstrates that while complete automation of transport planning may not be desirable or feasible, thoughtfully designed systems can significantly enhance planner capabilities.

ities. The success of the implementation at Van Berkel Logistics, and the subsequent founding of Squid, shows that academic research can translate directly into practical business value.

The future of transport planning lies not in replacing human expertise but in enhancing it with powerful tools that handle routine optimisation while leaving both data management and strategic decisions in human hands. As the logistics sector faces increasing complexity and pressure, such human-centric automation will become ever more crucial.

Bibliography

- [1] Luca Adacher and Marco Tiriolo. Performance analysis of decentralized vs centralized control for the traffic signal synchronization problem. *Journal of Advanced Transportation*, 2020:1–19, 2020.
- [2] World Bank and IAPH. The container port performance index 2021: A comparable assessment of container port performance, 2022. Accessed: 2025-05-08.
- [3] Mei-Shiang Chang, Che-Fu Hsueh, and Ph Student. Real-time vehicle routing problem with time windows and simultaneous delivery/pickup demands. *Journal of the Eastern Asia Society for Transportation Studies*, 5, 01 2003.
- [4] John Davin and Pragnesh Jay Modi. Impact of problem centralization in distributed constraint optimization algorithms. In *Proceedings of the Fourth International Joint Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 1057–1066, 2005.
- [5] Yorick C. de Vries. Reinforcement learning for order distribution in self-organizing logistics. Master’s thesis, Delft University of Technology, August 2021. Accessed: 2025-05-06.
- [6] DHL. Global supply chain woes deepen after suez canal blockage, 2021. Accessed: 2025-05-06.
- [7] Roberto Dominguez and Salvatore Cannella. Insights on multi-agent systems applications for supply chain management. *Sustainability*, 12(5), 2020.
- [8] Edmund H. Durfee, Victor R. Lesser, and Daniel D. Corkill. Trends in cooperative distributed problem solving. *IEEE Transactions on Knowledge and Data Engineering*, 1(1):63–83, 1989.
- [9] Sectorinstituut Transport en Logistiek. Beroepscompetentieprofiel planner transport, 2018. Accessed: 2025-05-06.
- [10] European Environment Agency. Monitoring progress of europe’s transport sector towards its environment, health and climate objectives, 2017. Accessed: 2025-05-06.
- [11] evofenedex. Co2-reductie in het goederenvervoer: regels en verplichtingen. Accessed: 2025-05-06.
- [12] Md Zahidur Farazi. Enhancing supply chain resilience with multi-agent systems and machine learning: a framework for adaptive decision-making. *The American Journal of Engineering and Technology*, 7:6–20, 03 2025.

- [13] Juliane Fischer, Christian Lieberoth-Leden, Johannes Fottner, and Birgit Vogel-Heuser. Design, application, and evaluation of a multiagent system in the logistics domain. *IEEE Transactions on Automation Science and Engineering*, page 1–14, 2020.
- [14] DHL Global Forwarding. Ocean freight market update, 2025. Accessed: 2025-05-06.
- [15] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, New York, NY, 1979.
- [16] Peter Godfrey-Smith. *Other Minds: The Octopus, the Sea, and the Deep Origins of Consciousness*. Farrar, Straus and Giroux, 2016.
- [17] Google. Geocoding api overview. Accessed: 2025-05-06.
- [18] Google. Google or-tools. Accessed: 2025-05-06.
- [19] Sacha Greif and Eric Burel. State of javascript: The annual developer survey of the javascript ecosystem. <https://stateofjs.com>, 2022. [Online; accessed 6-May-2025].
- [20] HERE Technologies. Geocoding & search api, 2025. Accessed: 2025-05-06.
- [21] John Hoxmeier and Chris Dicesare. System response time and user satisfaction: An experimental study of browser-based applications. *Proceedings of the Association of Information Systems Americas Conference*, 01 2000.
- [22] International Road Transport Union (IRU). Widening age chasm compounds truck driver shortage crisis: new iru report, April 2025. Accessed: 2025-05-06.
- [23] S. M. A. Bin Al Islam and Ali Hajbabaie. Distributed coordinated signal timing optimization in connected transportation networks. *Transportation Research Part C: Emerging Technologies*, 80:272–285, 2017.
- [24] Erkan Kayacan, Joshua M. Peschel, and Erdal Kayacan. Centralized, decentralized and distributed nonlinear model predictive control of a tractor-trailer system: A comparative study. In *Proceedings of the 2016 American Control Conference (ACC)*, pages 4403–4408, 2016.
- [25] N.A. Lang, J.M. Moonen, F. Srour, and Rob Zuidwijk. Multi agent systems in logistics: A literature and state-of-the-art review. *Erasmus Research Institute of Management (ERIM)*, *ERIM is the joint research institute of the Rotterdam School of Management, Erasmus University and the Erasmus School of Economics (ESE) at Erasmus Uni*, Research Paper, 01 2008.
- [26] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization, 2023.
- [27] Fang Liu. A greedy algorithm for solving ordinary transportation problem with capacity constraints. *Operations Research Letters*, 45(4):388–391, 2017.
- [28] Brendan Matkin. Safe sveltekit stores for SSR, May 2023. Accessed: 2025-05-08.

- [29] Bashir Naderi, Shadi Rahmani, and Shabnam Rahmani. A multiobjective iterated greedy algorithm for truck scheduling in cross-dock problems. *Journal of Industrial Engineering*, 2014:1–12, 05 2014.
- [30] Jakob Nielsen. Response times: The 3 important limits, 1993. Published by Nielsen Norman Group.
- [31] OpenTripModel. Opentripmodel, 2025. Accessed: 2025-05-09.
- [32] Joanna Partridge. Terrible timing and huge uncertainty: how ports and shipping are handling tariff turmoil. *The Guardian*, April 2025. Accessed: 2025-05-06.
- [33] Geert L. J. Pingen, Christian R. van Ommeren, Cornelis J. van Leeuwen, Ruben W. Fransen, Tijmen Elfrink, Yorick C. de Vries, Janarthanan Karunakaran, Emir Demirović, and Neil Yorke-Smith. Talking trucks: Decentralized collaborative multi-agent order scheduling for self-organizing logistics. In *Proceedings of the 32nd International Conference on Automated Planning and Scheduling (ICAPS 2022)*, pages 480–489. Association for the Advancement of Artificial Intelligence (AAAI), 2022. Accessed: 2025-05-06.
- [34] PostgreSQL Global Development Group. Postgresql: The world’s most advanced open source relational database.
- [35] Project OSRM Contributors. Open source routing machine (osrm): Modern c++ routing engine for shortest paths in road networks. Accessed: 2025-05-06.
- [36] Xiong Qiang, Martinson Yeboah Appiah, Kwasi Boateng, and Frederick VonWolff Appiah. Route optimization cold chain logistic distribution using greedy search method. *OPSEARCH*, 57:1115–1130, 2020.
- [37] Rijksoverheid. Rijttijden en rusttijden wegvervoer. [Accessed: 6 May 2025].
- [38] Adriano S. Silva, Jos Lima, Adrian M. T. Silva, Helder T. Gomes, and Ana I. Pereira. Time-dependency of guided local search to solve the capacitated vehicle routing problem with time windows, 2024.
- [39] Stichting Transport en Logistiek Nederland (STL). Ontwikkelingen arbeidsmarkt transport en logistiek. Accessed: 2025-05-06.
- [40] Svelte Team. Sveltekit documentation, 2025. Accessed: 2025-05-06.
- [41] A. N. Tarău, Bart De Schutter, and Hans Hellendoorn. Centralized, decentralized, and distributed model predictive control for route choice in automated baggage handling systems. *Control Engineering and Applied Informatics*, 11(3):24–31, 2009.
- [42] Christian van Ommeren and Elisah van Kempen en Jaco van Meijeren. Talking trucks: Zelforganiserende logistiek voor betrouwbaar vervoer. Technical report, SmartPort, 2022. Accessed: 2025-05-06.
- [43] C.R. van Ommeren, R.W. Fransen, G.L.J. Pingen, C.J. van Leeuwen, J.P. Paardekooper, and J.C. van Meijeren. Letting digital twins run the show: Exploring possibilities of letting vehicles plan and organise transportation themselves. Technical report, Netherlands Organisation for Applied Scientific Research (TNO), 2020. Accessed: 2025-05-06.

- [44] Yizhun Wang. Review on greedy algorithm. *Theoretical and Natural Science*, 14:233–239, 11 2023.
- [45] Dedy Rahman Wijaya, Aqil Athallah, Tiara Nuha Noor’afina, Patrick Adolf Telnoni, and Sari Dewi Budiwati. Cargo route optimization using shortest path algorithms: Runtime and validity comparison. *Journal of Computer Science*, 19(11):1369–1379, Oct 2023.
- [46] Martin Wolck and Stephan Meisel. Branch-and-pipe approaches for real-time vehicle routing with pickup, loading, and soft time windows. *Inform Journal on Computing*, 34, 2022.

A Appendix

A.1 Letter of Consent

The next page contains the anonymised letter of consent signed by Van Berkel Logistics for usage of their data.

Oxford University
Department of Computer Science
Professional Masters Programme in Software Engineering
Wolfson Building, Parks Road, Oxford, OX1 3QD
United Kingdom

20 December 2024

Michel van Dijk
Van Berkel Logistics
Marconiweg 1
5466 AS Veghel
Netherlands

Subject: Approval for the Use of Anonymised Data in Research on Algorithmic Decision Support for Container Road Transportation

Dear Michel van Dijk,

I am writing to formally request your approval for the use of anonymised data from Van Berkel Logistics for our research project entitled "Algorithmic Decision Support for Container Road Transportation." This research will investigate how best to support planners in making decisions regarding truck and driver combinations for container transportation.

The project will be conducted in Rotterdam, the Netherlands, in 2025 and aims to achieve the following high-level goals:

1. Capture the problem in a fitting data model
2. Design an appropriate software architecture
3. Explore and evaluate algorithmic solutions for automating planning proposals

To achieve these objectives, we will collect anonymised data regarding trucks, drivers, chassis, locations, and transports for several planning days. We will design and evaluate two algorithms—a centralised algorithm and a multi-agent decentralised algorithm—and compare their outputs against benchmarks based on decisions made by human planners.

Data Anonymisation Measures

To ensure the privacy and confidentiality of individuals and entities, the following anonymisation measures will be applied to the data before handing it over to the researcher (Christian van Ommeren):

- Driver names will be replaced with random names.
- Driver emailaddresses will be deleted.
- Truck license plates will be replaced by random characters.
- Chassis license plates will be replaced by random characters.
- Location names, addresses, phone numbers, emailaddresses and websites of will be deleted.

- Location code identifiers will be replaced by random characters when this location is a customer (codes of container terminals will remain as is).
- Transport datetimes will be shifted to a random date (e.g., data collected on May 15, 2024, will have timestamps shifted by an arbitrary number of days).

Please note that GPS-coordinates of locations will remain in the dataset to allow for accurate calculations of driving times. While these coordinates may indirectly identify customer locations, their inclusion is critical for achieving the objectives of this research. A full list of included fields and their modifications is included in the attachment.

Ethical Considerations

This research complies with the UK General Data Protection Regulation (UK GDPR) and the Data Protection Act 2018. All data will be handled in accordance with the University's Guidance on Data Protection and Research. Participants will be fully informed of the purpose of the study, the anonymisation measures undertaken, and any limitations on their ability to withdraw data once it has been anonymised or published.

Informed Consent Process

The intent of this letter is to obtain informed consent from Van Berkel Logistics for using anonymised planning data for our research project as specified above. If anything is unclear about the research, the anonymisation process, or the data itself you have the opportunity to ask questions and seek clarification prior to giving consent. Consent will be recorded and securely stored. Note that you may withdraw your data at any point before anonymisation or publication.

By signing below, you confirm your approval for the use of anonymised data from Van Berkel Logistics for this research.

Thank you for your support and cooperation. Should you have any questions or require additional information, please do not hesitate to contact me at christian.vanommeren@kellogg.ox.ac.uk or +31(0)611436002.

Yours sincerely,

Christian van Ommeren
MSc Software Engineering candidate
Oxford University

Approved by:

Michel van Dijk
Director Logistics
Date: _____

23-12-2024

A.2 Decision Ethics Committee

Worktribe™ ⋮

Home Profiles Ethics Help

 Ethics Application: Multi-Agent Matchmaking Between Trucks, Drivers, And Containerised Cargo (Archived)

☐ Not Required ⋮

Summary Details Scope Documents Versions Comments

Details

Application ID	964024
Title	Multi-Agent Matchmaking between Trucks, Drivers, and Containerised Cargo (Archived)
Project Description	Project for MSc in Software Engineering that investigates automating the planning process of container road transportation. For benchmarking purposes and to stay as close to reality as possible we would like to obtain real-life data which includes information about drivers and their whereabouts. Before using this data in research-setting we will obfuscate the data.
Org Unit	Computer Science
Project Dates	1 Jan 2024 - 9 May 2025
Is this a Student Project?	Yes - Taught Masters
Supervisor	Peter Bloodsworth
Applicant	
Submitter	
Visibility	People involved in this Application
Ethics Officer	Computer Science Ethics Officers
Status	☐ Not Required
Assigned To	 Peter Bloodsworth
Created	3 Dec 2024, 08:44
Created By	

Figure A.1: Decision Ethics Committee

A.3 Project Steps

All steps needed to complete the project are visualised in Figure A.2.

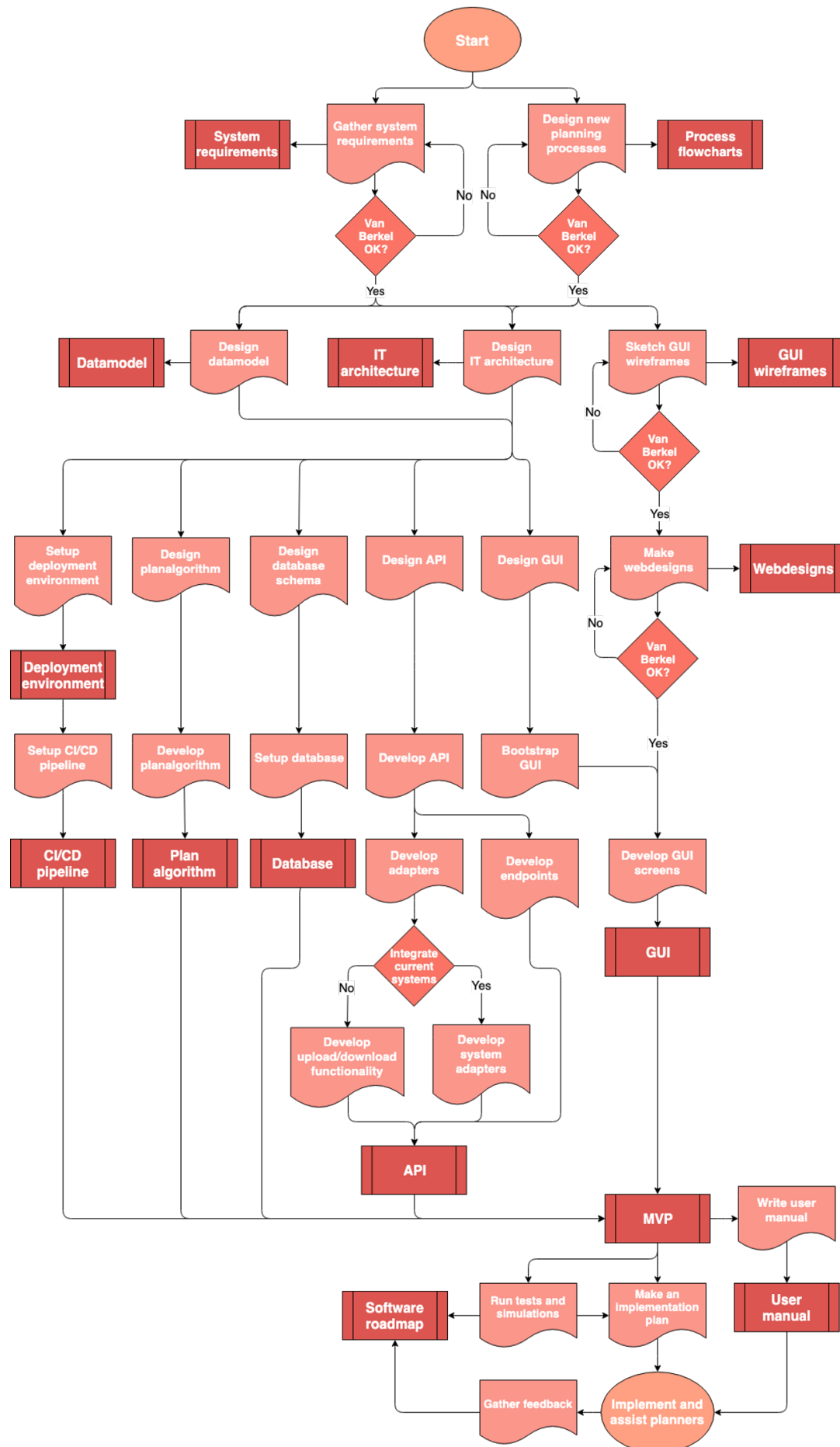


Figure A.2: Project Steps

A.4 Process Diagram

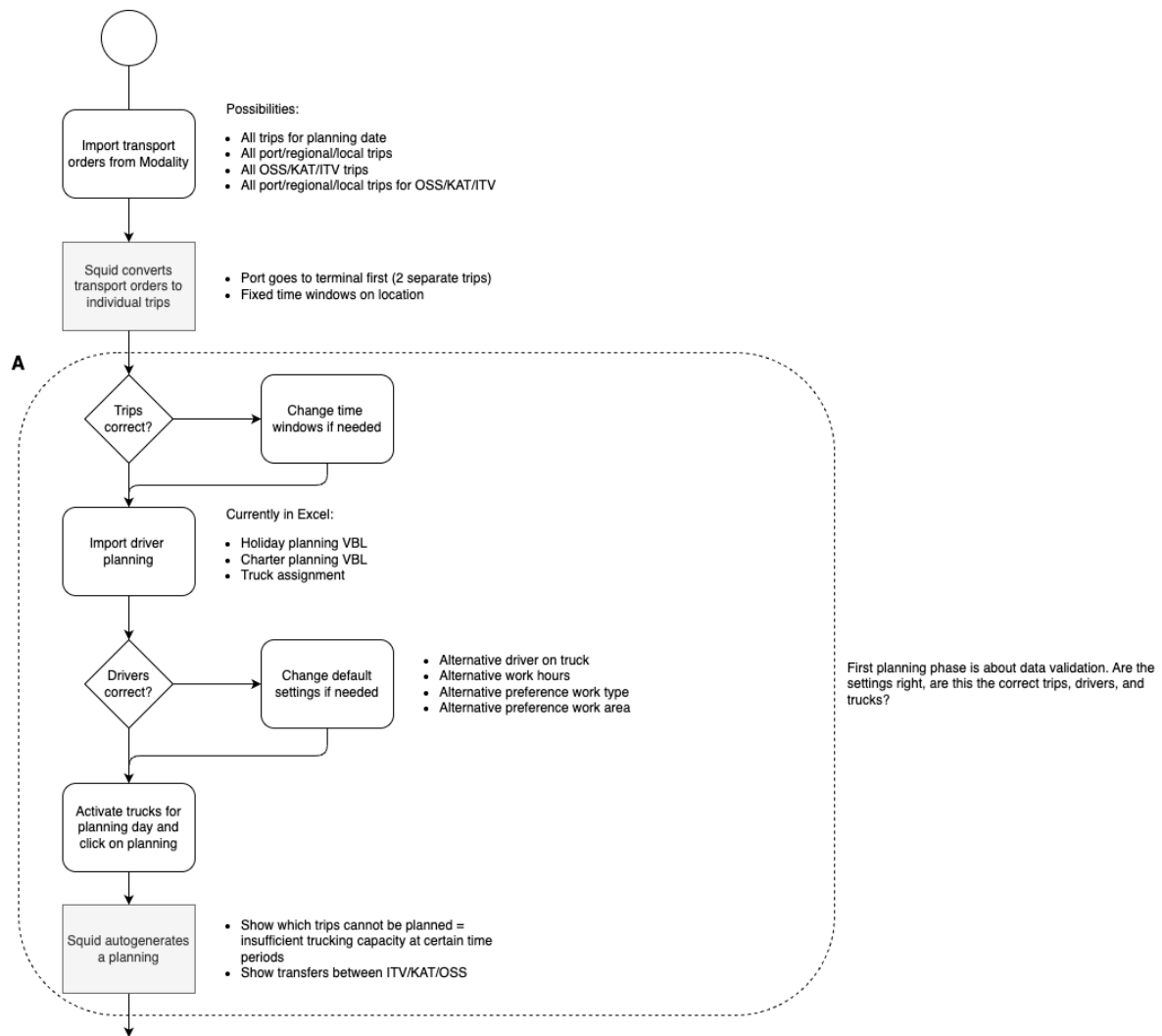


Figure A.3: Process Diagram - Phase A

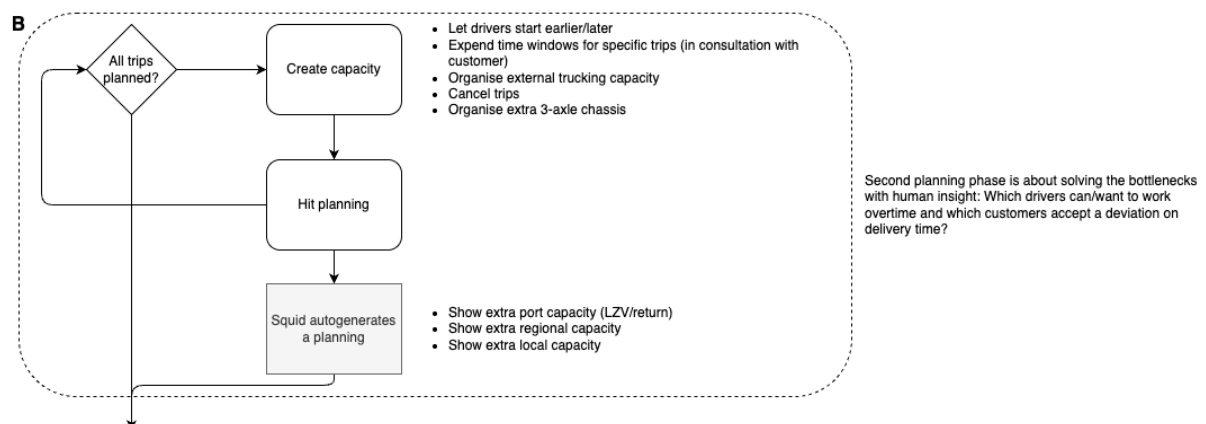


Figure A.4: Process Diagram - Phase B

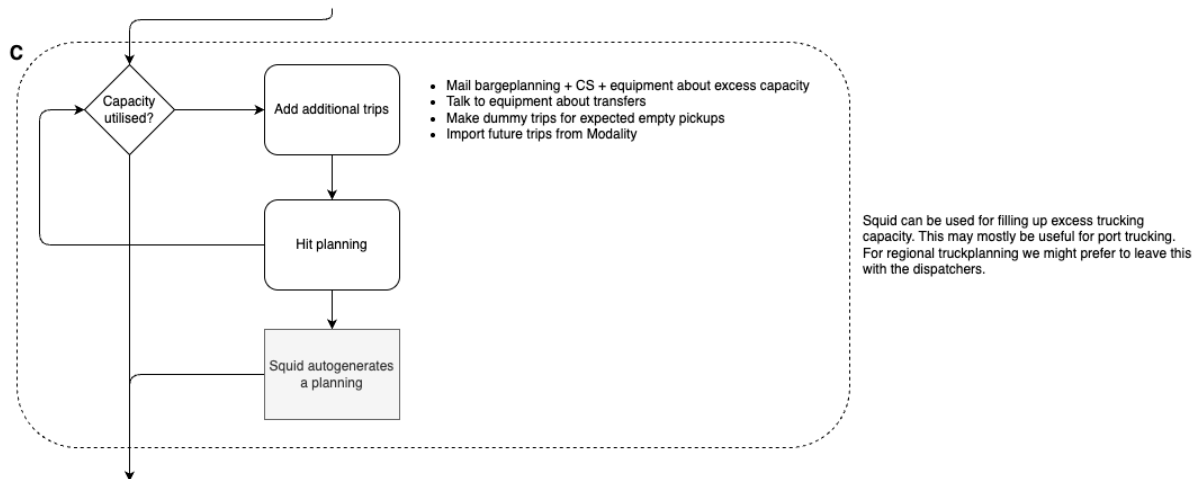


Figure A.5: Process Diagram - Phase C

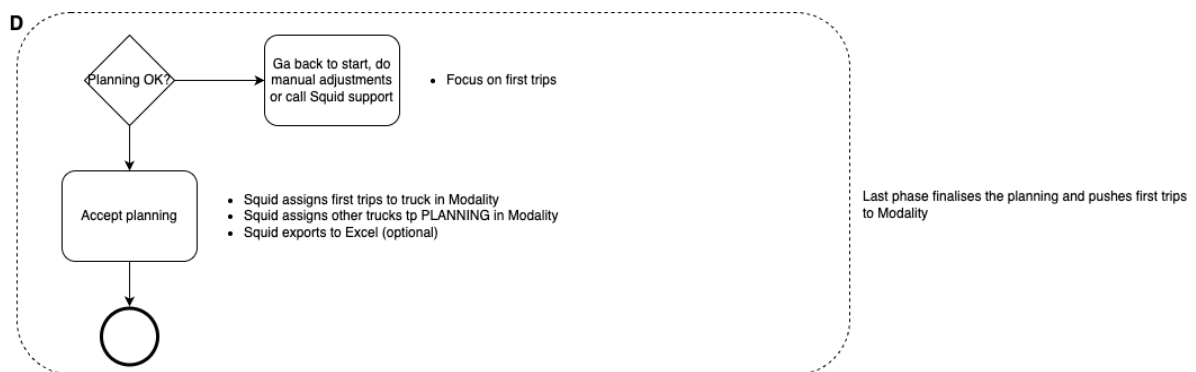


Figure A.6: Process Diagram - Phase D

A.5 User Story

A.5.1 Name

Day Planner User Story

A.5.2 Description

The day planner is responsible for:

- Mapping out transportation capacity
- Assigning first trips for seaport and regional drivers
- Creating a shared daily schedule per terminal for the terminal drivers

A.5.3 Actors

- Day Planner
- CS (Customer Support)
- Customers

- Drivers
- Barge Planners

A.5.4 Preconditions

- Customers have made bookings via CS
- CS has entered all truck orders into Modality
- Drivers have submitted their availability and working hours, recorded by planners
- CS and planners have agreed with customers on acceptable time window deviations
- Planners have recorded preferences together with the drivers

A.5.5 Standard Flow

1. Planner opens Squid
2. Planner selects a planning day
3. Planner imports all trips from Modality scheduled for that day
4. Planner checks the trips
5. Planner checks the available drivers
6. Planner activates the trucks
7. Planner clicks on *Planning*
8. Squid generates a plan
9. Planner accepts the plan

A.5.6 Alternative Flows

Driver data for planning day is incorrect

- Planner modifies driver data (for this specific day)

New address in Modality, standard time windows used

- Planner adjusts time windows (in consultation with CS) for the new customer

Trips are incorrect (e.g. seaport must be direct)

- Planner adjusts the incorrect trip

Planner disagrees with the planning

Insufficient transportation capacity

- Planner adjusts driver working hours, engages extra charter, changes time windows, or cancels trips
- Planner clicks *Planning* again (Step 7)

Surplus transportation capacity

- Planner imports extra trips from Modality (after consultation with CS, barge planning, and equipment) or cancels a charter
- Planner clicks *Planning* again (Step 7)

*Exception Flows***Squid generates an illogical plan**

- Planner informs Squid
- Planner manually forces required changes or reverts to Excel export
- Planner optionally clicks *Planning* again (Step 7)

Required change is impossible in Squid

- Planner informs team lead
- Team lead adds required change to the feature list
- Squid adds the feature after consultation

A.5.7 Result

- Squid places first trips on license plate in Modality, and remaining trips in PLANNING

A.6 Wireframes

[illegible]

https://squidsoftware/chauffeurs/rooster

Squid Truckplanning

Chauffeurs

Trucks

Ritten

Planbord

Uitloggen

zoeken

Nieuwe truck toevoegen

Trucks

Standaard

Vandaag

Morgen

Overmorgen

Er is 1 truck actief zonder chauffeur!

Standplaats

ITV

Euronorm

6

Werkgebied

Regio

Actief

Truck	Chauffeur	Euro	Weg	DE	BE	LZV	ADR	Slaapcabine	Voorloopas	Hefschotel	Actief	Acties
V1	Michel van Dijk	6										
V2	Patrick Dammers	6										
V11	Marco van Beers	6										
57-RTZ-8	Vaste chauffeur	6										

Nieuwe truck toevoegen

[illegible]

[illegible]

A.7 Designs

[illegible]

Dag planner

Dag opvolger

+ Nieuwe chauffeur toevoegen

Chauffeursrooster

Standard Vandaag Morgen Overmorgen 

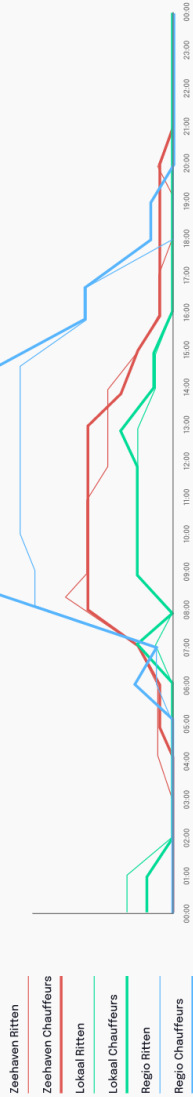
Standplaats Alle 

Werktijden

08:00 - 17:00 

Werkgebied

Regio 



Michiel van Dijk																								
Andere naam																								
Michiel van Dijk																								
Andere naam																								

Dag planner

Dag opvolger

Trucks

+ Nieuwe Truck toevoegen

Standard

Vandaag

Morgen

Overmorgen

...

Standplaats

ITV

Euronorm

6

Werkgebied

Regio

Actief

Actief	Truck	Chauffeur	Standplaats	Euro	Weg	DE	BE	LTV	ADR	Slaapcabine	Hefschotel	Bewerk
	V1	Michiel van Dijk	ITV	6								
	V2	Patric Dammers	ITV	6								
	V3	Marco van Beers	ITV	6								
	V1	Michiel van Dijk	ITV	6								
	V2	Patric Dammers	ITV	6								
	V3	Marco van Beers	ITV	6								
	V1	Michiel van Dijk	ITV	6								
	V2	Patric Dammers	ITV	6								
	V3	Marco van Beers	ITV	6								
	V1	Michiel van Dijk	ITV	6								

Dag planner

Dag opvolger

Ritten

Ritten exporteren

+ Nieuwe Ritten toevoegen

Standaard

Vandaag

Morgen

Overmorgen

...

Zeeken

Klant / Contactnr. / Booking ID

Q

Status

Alle

Terminal

ITV

Truck

Alle

Tijdstip

Service

Werkgebied

Alle

BookingID	Containernr	Status	Truck	Type	Gewicht	Richting	Route	Klant	Tijdstip	Service	Douane	Terminal	Rederij	Blockade	Container	Gasmetheding	Bewerk
20123A	MSC12345678	Gepland	V1	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Beschikbaar	-	40HC	50.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V2	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V1	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Beschikbaar	-	40HC	50.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V2	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V1	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Beschikbaar	-	40HC	50.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V2	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V1	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Beschikbaar	-	40HC	50.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V2	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	
20123A	MSC12345678	Gepland	V1	40HC	30.000	Export		Mars	10:00	Live handling	X	X	X	X	X	X	

[illegible]

A.8 Application Screenshots

The screenshot displays the 'Chauffeursafspraken' (Driver Appointments) application. The interface is divided into a sidebar on the left and a main content area on the right.

Sidebar:

- Top: 1.5.22, Overview icon, Planbord, Ritten, Containers, Chauffeurs (selected), Afspraken (active), Roosters, Trucks, Chassis, Locaties, Profiel, Instellingen, Syncfouten, Uitloggen.
- Bottom: Vandaag, Morgen, 01/05/2025.

Main Content Area:

Header: Chauffeursafspraken

Filters: Naam, Q, Standplaats, Alle, Werkgever, Alle, LZV, ADR, Tank ADR, Acceptabel overwerk, Voorkeur servicetype, Alle, Slaapprookeur, Alle, Acties.

Naam	Standplaats	Werkgever	LZV	ADR	Tank ADR	Acceptabel overwerk	Voorkeur servicetype	Slaapprookeur	Voorkeur werkgabied	Acties
Alexander	VEG	VEG	X	X	X	Voor: 15 min Na: 120 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 4% Regio: 48% Zeehaven: 48%	
Amber	VEG	UXUP	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 0% Regio: 100% Zeehaven: 0%	
Anira	VEG	PEMUNJ	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 0% Regio: 100% Zeehaven: 0%	
Anika	VEG	VEG	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 0% Live handling: 100%	Thuis: 100% Auto: 0%	Terminal: 95% Regio: 5% Zeehaven: 0%	
Anna	KAT	YOFAMUX	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 10% Regio: 90% Zeehaven: 0%	
Anouk	KAT	MAKAMEX	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 10% Regio: 90% Zeehaven: 0%	
Arie	OSS	EYIZEKUM	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 0% Regio: 100% Zeehaven: 0%	
Arjan	OSS	UZUTIP	X	X	X	Voor: 60 min Na: 60 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 0% Regio: 100% Zeehaven: 0%	
Ayia	VEG	VEG	X	X	X	Voor: 60 min Na: 30 min	Afroppeken: 50% Live handling: 50%	Thuis: 100% Auto: 0%	Terminal: 25% Regio: 75% Zeehaven: 0%	

WhatsApp icon in the top right corner.

15.22

Overzicht

Planbord

Ritten

Containers

Chauffeurs

Afspraken

Roosters

Trucks

Chassis

Locaties

Profil

Instellingen

Syncfouten

Uitloggen

Squad Truckplanning - Trucks

oxford.localhost:3000/trucks?date=2025-05-01

Vandaag

Morgen

01/05/2025

Trucks

Donderdag 1 mei 2025

Kenteken

Standplaats

Eigenaar

Beschikbaar

Afwijkingen

TT10 TT11

TT11 Beschikbaar

RI1 Beschikbaar

RI2 Beschikbaar

Beschikbaar	Nummer	Standplaats	Eigenaar	Euronorm	Openbare weg	OBU Duitsland	OBU België	LZV	ADR aansluiting	Voorloopas	Slaapcabine	Hefschotel	Acties
	TT10	KAT	VEG	0									
	TT11	KAT	VEG	0									
	TT9	KAT	VEG	0									
	V19	KAT	VEG	5									
	V38	KAT	VEG	6									
	V41	KAT	VEG	6									
	V52	KAT	VEG	6									
	V60	KAT	VEG	6									
	V64	KAT	VEG	6									
	V31	OSS	VEG	6									
	V44	OSS	VEG	6									
	V46	OSS	VEG	6									
	V61	OSS	VEG	6									
	RI2	RIKLUXI	VEG	0									
	TT15	TIEL	VEG	6									
	TT16	TIEL	VEG	0									
	TT17	TIEL	VEG	0									
	V66	TIEL	VEG	0									

81

Squid Truckplanning - Localities

X +

oxford.localhost:3000/locations?date=2025-05-01&country=BE

Vandaag Morgen 01/05/2025

Locaties

Donderdag 1 mei 2025

Code

Q Naam

Q Adres

Q Servicetype

Allé

Minimale euronorm

Allé

Land

België

Code

Type

Adres

Land

Gasmeting

In/uitwegen

Oprijtijden

Acceptabel te vroeg

Maximaal te laat

Minimale euronorm

Acties

+	ASSOCIA	Associated Terminal Operators	Zeehaven	Kaal 550 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	BECOMAR	Belgian Container Maintenance en Repair NV	Zeehaven	Antwerpsebaan 1 B-2040 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	DOORNANT	Van Doorn	Zeehaven	Muisbroeklaan 34A (K504) 2030 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	K118	Becomar NV / Zuid Nulle	Zeehaven	Nauwkestraat 2030 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	K234	kaal 234	Zeehaven	Kragenvweg 4 2030 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	K734	Becomar NV K734	Zeehaven	Antwerpsebaan 2040 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	K740	DP wacs kaal 740	Zeehaven	Nieuwe Westweg - Haven 740 B2040 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	MEDREP4	Medrepair 4 Antwerpen	Zeehaven	Muisbroeklaan 2030 - Kaal 524 B2030 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	MEGAND	Mega industrie Engineering	Zeehaven	Treurenborg 7/K420 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	PROGA	Progeco Belgium 2	Zeehaven	Transcontinentalweg 2 2030 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	
+	PROGA2	Progeco Belgium 2	Zeehaven	Treurenborg 1-9 Kaal 426 B2030 Antwerpen		x	x	00:00 - 23:59	-15	15	-30	30	0	

Chauffeurs

Afspraken

Roosters

Trucks

Chassis

Localities

Profil

Instellingen

Synctouten

Uitloggen

1.5.22

Overzicht

Planbord

Ritten

Containers

1.5.22

Overzicht

Planbord

Ritten

Containers

Chauffeurs

Afspraken

Roosters

Trucks

Chassis

Localities

Profil

Instellingen

Syncfouten

Uitloggen

Squid Truckplanning - Containers

oxford.local:3000/containers

Containers ophalen

Transportboekingen

Zoek op boekingscode, containernummer, of locatiecode

Code

Code	Laad/los adres	Containernummer	Containertype	Route	First pickup	Last pickup	Laad/los datum	Cargo opening	Cargo closing
2945148	UYONAMA	-	-	UYONAMA > VEG ECTDDE > VEG VEG > FRISO VEG > UYONAMA	27/12 00:10	23/12 10:40	30/12 07:00	-	04/01 00:00
294667A	IDOJIXUB	-	-	HPD2 > KAT KAT > IDOJIXUB IDOJIXUB > KAT	17/12 07:00	20/12 07:00	28/02 07:30	-	-
294873A	OYEMEG	-	-	VEG > OYEMEG > VEG APMR2 > VEG	15/02 13:00	17/02 20:59	24/02 14:00	-	-
294873B	OYEMEG	-	-	VEG > OYEMEG > VEG APMR2 > VEG	15/02 13:00	17/02 20:59	25/02 08:00	-	-
294873C	OYEMEG	-	-	VEG > OYEMEG > VEG APMR2 > VEG	15/02 13:00	17/02 20:59	25/02 10:00	-	-
294873E	OYEMEG	-	-	APMR2 > VEG VEG > OYEMEG > VEG	15/02 13:00	17/02 20:59	25/02 12:00	-	-
294873G	OYEMEG	-	-	APMR2 > VEG VEG > OYEMEG > VEG	15/02 13:00	17/02 20:59	24/02 10:00	-	-
294873H	OYEMEG	-	-	VEG > OYEMEG > VEG APMR2 > VEG	15/02 13:00	17/02 20:59	24/02 08:00	-	-
295313A	XAYEG	-	-	VEG > K913 K913 > VEG VEG > XAYEG > VEG	20/02 00:00	11/03 00:00	27/03 08:00	-	03/04 00:00
295990C	IDOJIXUB	-	-	EMX > KAT KAT > IDOJIXUB > VEG	16/01 23:59	18/01 21:55	26/02 08:00	-	-
296044A	OYEMEG	-	-	OSS > OYEMEG > OSS ECTDDE > OSS	17/03 00:00	18/03 23:59	24/03 08:00	-	-
296044B	ORAMOL	-	-	ECTDDE > OSS OSS > ORAMOL > VEG	17/03 00:00	18/03 23:59	24/03 07:00	-	-
296217A	WOFOVE	-	-	EMX > TIEL WOFOVE > TIEL TIEL > WOFOVE TIEL > EMX	16/02 00:01	17/02 19:01	26/02 08:00	-	09/03 00:00
296217B	WOFOVE	-	-	TIEL > EMX EMX > TIEL TIEL > WOFOVE WOFOVE > TIEL	16/02 00:01	17/02 19:01	25/02 09:00	-	09/03 00:00
296217C	WOFOVE	-	-	WOFOVE > TIEL TIEL > EMX TIEL > WOFOVE EMX > TIEL	16/02 00:01	17/02 19:01	26/02 09:00	-	09/03 00:00
296217D	WOFOVE	-	-	EMX > TIEL TIEL > EMX TIEL > WOFOVE WOFOVE > TIEL	16/02 00:01	17/02 19:01	27/02 07:00	-	09/03 00:00
296220A	MAWUTAWU	-	-	TIEL > EMX MAWUTAWU > TIEL EMX > TIEL	16/02 00:01	17/02 19:01	26/02 07:00	-	03/03 00:00
296220B	MAWUTAWU	-	-	MAWUTAWU > TIEL TIEL > MAWUTAWU TIEL > EMX EMX > TIEL	16/02 00:01	17/02 19:01	27/02 05:00	-	03/03 00:00

[illegible]

[illegible]